

Strong normalization through idempotent intersection types: a new syntactical approach ^{*†}

Pablo Barenbaum^{3,2} Simona Ronchi Della Rocca¹ Cristian Sottile^{2,3}
pbarenbaum@dc.uba.ar ronchi@di.unito.it csottile@dc.uba.ar

¹Dipartimento di Informatica, Università di Torino, Italy

²ICC, CONICET–Universidad de Buenos Aires, Argentina

³Universidad Nacional de Quilmes, Argentina

Abstract

It is well-known that intersection type assignment systems can be used to characterize strong normalization (SN). Typical proofs that typable lambda-terms are SN in these systems rely on *semantical* techniques. In this work, we study $\Lambda_{\cap}^e$, a variant of Coppo and Dezani’s (Curry-style) intersection type system and we propose a syntactical proof of strong normalization for it. We first design $\Lambda_{\cap}^i$, a Church-style version of $\Lambda_{\cap}^e$, in which terms closely correspond to typing derivations. We then prove that typability in $\Lambda_{\cap}^i$ implies SN through a measure that, given a term, produces a natural number that decreases along with reduction. This measure provides a *syntactical* proof of SN. Finally, the result is extended to $\Lambda_{\cap}^e$, since the two systems simulate each other.

1 Introduction

Intersection types were introduced by Coppo and Dezani [12], with the explicit aim of increasing the typability power of simple types. Beyond the arrow connective of simple types ($\rightarrow$), these systems incorporate an intersection connective ($\wedge$), which is commutative, associative and idempotent (*i.e.* $A \wedge A \equiv A$), which means that intersection is essentially a notation for *sets* of types.

Intersection type assignment systems come in different versions, and are able to characterize semantical properties of terms. For example, Coppo and Dezani’s original system *characterizes* strong normalization, in the sense that a term is typable if and only if it is strongly normalizing. Recall that a λ -term is *strongly normalizing* if every reduction sequence starting from it eventually terminates.

There are many other intersection type systems characterizing other properties besides strong normalization, such as head normalization [13, 24], where we recall that a λ -term is *head normalizing* if head reduction (reducing at every step the redex in head position) eventually terminates. Furthermore, intersection types are a very powerful tool to reason about the interpretation of terms in various models of λ -calculus [4, 30, 5].

The existent proofs of the characterization properties are not straightforward: for both strong and head normalization, the direction “typable implies normalizing” is generally carried out using semantical tools like computability or reducibility candidates [29, 24]. These semantical proofs are in some sense standard, but it is not easy to extract from them an intuition about what is going on. Having an explicit measure that decreases with β -reduction would be preferable. In the case of strong normalization, there are three completely syntactical proofs, based on a decreasing measure [22, 9, 11]; we will briefly discuss them and their relation with the present work at the end of this section.

A non-idempotent version of intersection types has been defined [20, 21, 15], where the intersection connective is *not* idempotent (*i.e.* $A \wedge A \not\equiv A$), and thus becomes a notation for *multisets* of types. The non-idempotent systems preserve some of the properties of the idempotent ones: they can characterize strong and

^{*}Funded by the European Union through the MSCA SE project QCOMICAL (Grant Agreement ID: 101182520).

[†]We would like to thank the anonymous reviewers for their careful reading of the manuscript and their insightful comments, which helped improve the quality of this paper.

head normalization [7]. From a semantical point of view, they can describe relational models of λ -calculus [28]; even if this class of models is quite poor, with respect to the λ -theories that they induce, they are useful to explore quantitative properties of typable terms. Unlike what happens in the idempotent case, the proof that these systems with non-idempotent intersection characterize strong and head normalization can be carried out by an easy induction on the size of the typing derivation, which decreases at every reduction step. This relies on the quantitative nature of non-idempotency, and in particular on the fact that, in a derivation, the cardinality of the multiset of types of a variable x is an upper bound of the number of occurrences of x in the typed term. Given that these proofs rely crucially on the fact that intersection is non-idempotent, there is not much hope of extending them to the idempotent case. But we think that it would be interesting to explore if there is a decreasing measure for idempotent intersection types which is simpler than those in [22, 9]. Here we present a positive answer to this problem, namely a decreasing measure consisting of just a natural number.

1.1 Contributions

Recently, a proof of strong normalization for simply typed λ -calculus has been presented [2], based on a decreasing measure which can be defined constructively, by relying on a non-erasing notion of reduction inspired by *memory calculi* [25, 23, 16, 27, 10]. Namely, each β -reduction step creates a “wrapper” containing a copy of the argument, remembering all the erased subterms. An operation called *full simplification* is defined to repeatedly perform the complete development of all the redexes of maximum degree in a term¹. A measure decreasing at every reduction step is then given simply by a natural number corresponding to the number of wrappers that remain after full simplification. Here we **adapt this technique to idempotent intersection types**.

The first step in order to do this job is to design an intrinsically typed version of intersection types, which are usually presented as extrinsically typed systems instead. Let us recall that, in an *extrinsically typed* (Curry-style) system, terms are untyped and there is a set of typing rules assigning types to them, whereas in an *intrinsically typed* (Church-style) system, terms are decorated with types, and typing rules are just a check of syntactical correctness. There are some attempts in the literature in this direction, *e.g.* in [8, 26, 1], but they are not suitable for the application of the discussed techniques. In fact the big difference between simple and intersection types, on which the power of the latter relies, is that, if M is a term of λ -calculus typable by intersection types and N is a subterm of M occurring once in it, a typing derivation of a judgement $\Gamma \vdash_e M : A$ can contain $n \geq 1$ subderivations with object N . Since N can contain redexes, performing a reduction inside N corresponds to performing n reductions *in parallel* on the derivation. The natural way to achieve this is to equip the Church-style intersection type system with a parallel notion of reduction, and this is indeed what all the existing proposals do. At the same time, in order to apply the cited method, we would need to reduce these subderivations independently from each other.

Our starting point is an extrinsically typed subset of Λ , called $\Lambda_\cap^e$, defined through an intersection type assignment system. This system is a variant of well-known systems in the literature that characterize strong normalization. In order to build an intrinsically typed version of $\Lambda_\cap^e$, we exploit the idea that idempotent intersection represents a set of types, and we explicitly use sets both in the grammar of types and terms of the typed language, so for example if t and s are terms of type A and B respectively, then $\{t, s\}$ is a set of terms which can be assigned the set of types $\{A, B\}$. Following this idea, we formulate an intrinsically typed version of $\Lambda_\cap^e$ called $\Lambda_\cap^i$, where sets of types are decorations for sets of terms. While these two systems can be dealt with independently from each other, they are shown to *simulate* each other. The new typed language $\Lambda_\cap^i$ enjoys all the good properties we expect, such as subject reduction and confluence. More importantly, it is suitable for the application of a method extending that of [2]. As a matter of fact, using the notion of complete development by degree, and relying on an auxiliary *memory calculus* $\Lambda_\cap^{im}$, we define a measure which is a natural number decreasing at every reduction step in $\Lambda_\cap^i$. Then, we prove that, for every term M in $\Lambda_\cap^e$, there is a term t in $\Lambda_\cap^i$ such that an infinite reduction sequence in M entails an infinite reduction sequence in t , thus obtaining a new proof of strong normalization for $\Lambda_\cap^e$. Finally, we complete the picture

¹In this context, the *degree* of a redex $(\lambda x. t) s$ is the height of the type of the abstraction $\lambda x. t$.

by proving that $\Lambda_{\cap}^{\dot{1}}$ gives type to all the strongly normalizing terms, hence showing that it has the same typability power of the original paper [12].

1.2 Comparison with related works

As we recalled before, there are three other syntactical proofs of strong normalization for idempotent intersection types, namely [22, 9, 11]. Both [22, 9] define an intermediate language, typed by intersection types, equipped with a suitable non-erasing reduction rule. So, as known for λI (*i.e.* non-erasing) based systems [3, §11.3], weak normalization there entails strong normalization. Using the property that the normalization of both the reductions implies β -normalization, they prove weak normalization for terms typable in the intersection type assignment system, and obtain strong normalization as a corollary. In both papers, the normalization proofs are carried out by defining a measure that decreases when a particular reduction strategy is chosen. The approach of reducing weak to strong normalization can be traced back to at least Nederpelt's and Klop's theses [25, 23].

In [22] the intermediate language is an intrinsically typed λ -calculus extended with lists, the reduction rule is the composition of a parallel β -rule and a commutation rule, and the measure is a multiset of natural numbers decreasing under the innermost reduction. In [9] the intermediate language is a variant of the Klop's calculus, extrinsically intersection typed, and the reduction is the β -reduction, but copying its argument when it is going to be erased; the weak normalization proof is obtained through the weak cut-elimination property of the type derivation, and the measure is a pair of natural numbers, decreasing under the strategy choosing the innermost redex of maximum degree.

Since our work uses an approach in some sense similar to [22, 9], we first highlight the novelties that we consider important with respect to them.

- We share with [22] the use of an intermediate language which is intrinsically typed by intersection types, but while their language is *ad hoc*, and its properties are not explored, $\Lambda_{\cap}^{\dot{1}}$ is shown here to be the Church version of the Curry-style system. Indeed, $\Lambda_{\cap}^{\circ}$ and $\Lambda_{\cap}^{\dot{1}}$ simulate each other, and reduction corresponds to cut-elimination in the derivation tree; we consider this to be an important result on its own.
- We do not need to define an *ad hoc* reduction rule; $\Lambda_{\cap}^{\dot{1}}$ uses directly the β -reduction, in its typed version.
- The codomain of our measure is simpler than multisets and pairs in [22, 9], consisting of a single natural number.
- Our technique provides a measure that decreases for *every* reduction strategy. There are proofs with these characteristics for the λ -calculus, such as Gandy's proof and its derivatives [19, 17], but not for intersection types, as far as we know.

Despite these differences, their works and ours rely fundamentally on Turing's remark that contracting a redex cannot create redexes of higher or equal degree [18, 6], which is the basis for Turing's proof of weak normalization of the simply typed λ -calculus [18].

The approach of [11] is completely different: indeed, the normalization of intersection types here is not the main result, but it is an intermediate step in the proof that computational power of intersection types is the same as that of simple types. The authors define an embedding from intersection type to simple type derivations which commutes with the β -reduction. So they do not define any decreasing measure. Let us notice that the embedding translates terms to linear terms, and that the intersection is used in a non commutative way, since the translation of $A \wedge B$ does not coincide with that of $B \wedge A$.

One could define a measure for intersection types by combining the translation with one of the existing measures for simply typed λ -calculus, for instance the measure $\mathcal{W}$ in [2]. Due to information lost in the translation, this would supply a natural number greater or equal than the one obtained in this paper. Since decreasing measures serve as an upper bound for the longest reduction chain of terms, a tighter measure provides a more accurate computational analysis and would therefore be generally preferred.

It is worth noting that even the combination with a simply typed λ -calculus exact measure (*e.g.* [17]) would not yield a tighter (nor looser) measure than $\mathcal{W}$. The loss of intersection type information that occurs during the translation is a hard limitation for any measure based on it; they cannot be refined to exactness. Our more coupled approach, since it operates *within* the intersection system, provides a finer analysis of reduction and thus allows for further improvement, which is indeed one of the proposed future works. We consider the refinability of our measure an advantage over measures based on such translation.

Organization of this paper Section 2 presents the languages $\Lambda_{\cap}^e$ and $\Lambda_{\cap}^i$, their relations and properties. Section 3 constructs a decreasing measure that entails strong normalization of $\Lambda_{\cap}^i$ by means of an auxiliary memory calculus $\Lambda_{\cap}^{im}$. Finally, Section 4 shows how this can be used to prove strong normalization of $\Lambda_{\cap}^e$, concludes, and proposes future works.

2 An intrinsically typed presentation of idempotent intersection types

This section starts by presenting $\Lambda_{\cap}^e$ (2.1), an intersection type assignment system for λ -terms, closely related to other well-known systems that characterize strong normalization. In Section 2.2, we formulate an intrinsically typed version of the system, called $\Lambda_{\cap}^i$, and we study its properties, including subject reduction and confluence. In Section 2.3 we establish a formal proof-theoretical connection between $\Lambda_{\cap}^e$ and $\Lambda_{\cap}^i$, and we prove that $\Lambda_{\cap}^e$ gives type to all the strongly normalizing terms. Recall that the set of untyped λ -terms Λ is defined as usual by the grammar $M ::= x \mid \lambda x. M \mid M M$. The sets of **types** $(A, B, \dots)$ and **set-types** $(\vec{A}, \vec{B}, \dots)$ are given mutually inductively as follows:

$$A ::= a \mid \vec{A} \rightarrow A \quad (\vec{A} \neq \emptyset) \quad \vec{A} ::= \{A_j\}_{j \in J}$$

where $a, b, \dots$ range over a denumerable set of base types, and $\vec{A}$ stands for a finite set of types², where J denotes a finite set of indices, and $\{A_j\}_{j \in J}$ is the set $\{A_j \mid j \in J\}$. When we write a set-type, we keep the invariant that there are no repetitions, *i.e.* that if $h \neq k$ then $A_h \neq A_k$, so that the set is of cardinality $|J|$. Sometimes we will use the explicit notation $\{A_1, \dots, A_n\}$. Note that a set-type is a finite set of types that may be empty, but the domain of an arrow type is always a *non-empty* set-type.

Sometimes we write $A \rightarrow B$ for $\{A\} \rightarrow B$.

A typing context $(\Gamma, \Delta, \dots)$ is a function mapping each variable to a set-type such that $\Gamma(x) \neq \emptyset$ for finitely many variables x . We write $\text{dom}(\Gamma)$ for the set of variables which Γ assigns to a non-empty set-type. We adopt the standard notation for typing contexts, *e.g.* writing $x_1 : \vec{A}_1, \dots, x_n : \vec{A}_n$ for the context Γ such that $\Gamma(x_i) = \vec{A}_i$ for each $i \in 1..n$ and $\Gamma(y) = \emptyset$ for every $y \notin \{x_1, \dots, x_n\}$. We write $\Gamma \cup \Delta$ for the typing context such that $(\Gamma \cup \Delta)(x) = \Gamma(x) \cup \Delta(x)$ for every x , and we write $\Gamma \subseteq \Delta$ if $\Gamma(x) \subseteq \Delta(x)$ for every x . Note in particular that $\Gamma \subseteq (\Gamma, x : \vec{A})$ if $x \notin \text{dom}(\Gamma)$. If $\mathcal{J}$ is a judgement, we write $\Pi \triangleright \mathcal{J}$ to mean that Π is a particular derivation proving $\mathcal{J}$.

2.1 Extrinsically typed system

We define a Curry-style intersection type assignment system, which is a variant of well-known systems in the literature, starting with the one introduced by Coppo and Dezani [12].

Definition 2.1 (The $\Lambda_{\cap}^e$ type assignment system). The $\Lambda_{\cap}^e$ calculus is defined by means of two forms of judgement:

1. $\Gamma \vdash_e M : A$ meaning that the λ -term M has type A under the context Γ .

²Note that $\vec{A} ::= \{A_j\}_{j \in J}$ is not intended to be a grammatical production that determines a concrete syntax for finite sets, but rather as a formation rule meaning that a set-type $\vec{A}$ is given by a finite set of types.

2. $\Gamma \Vdash_e M : \vec{A}$ meaning that the λ -term M has set-type $\vec{A}$ under the context Γ .

The typing rules are as follows:

$$\frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_e x : B} \text{ e-var} \quad \frac{(\Gamma \vdash_e N : A_j)_{j \in J} \quad (\forall h, k \in J. h \neq k \implies A_h \neq A_k)}{\Gamma \Vdash_e N : \{A_j\}_{j \in J}} \text{ e-many}$$

$$\frac{\Gamma, x : \vec{A} \vdash_e M : B}{\Gamma \vdash_e \lambda x. M : \vec{A} \rightarrow B} \text{ e-I} \rightarrow \quad \frac{\Gamma \vdash_e M : \vec{A} \rightarrow B \quad \vec{A} \neq \emptyset \quad \Gamma \Vdash_e N : \vec{A}}{\Gamma \vdash_e M N : B} \text{ e-E} \rightarrow$$

Note that we restrict the codomain of contexts so that $\Gamma(x) \neq \emptyset$, so rule **e-I** $\rightarrow$ cannot introduce an empty set to the left of the arrow.

Example 2.2. Let $\Gamma = x : \{\{A, B\} \rightarrow C, A, B\}$. Then:

$$\frac{\frac{\{A, B\} \rightarrow C \in \Gamma(x)}{\Gamma \vdash_e x : \{\{A, B\} \rightarrow C\}} \text{ e-var} \quad \frac{\frac{A \in \Gamma(x)}{\Gamma \vdash_e x : A} \text{ e-var} \quad \frac{B \in \Gamma(x)}{\Gamma \vdash_e x : B} \text{ e-var}}{\Gamma \Vdash_e x : \{A, B\}} \text{ e-many}}{\Gamma \vdash_e x x : C} \text{ e-E} \rightarrow$$

$$\frac{\Gamma \vdash_e x x : C}{\emptyset \vdash_e \lambda x. x x : \{\{A, B\} \rightarrow C, A, B\} \rightarrow C} \text{ e-I} \rightarrow$$

Remark 2.3. The system $\Lambda_\cap^\circ$ presented above is not exactly the same system as that of [12], for two reasons. First, $\Lambda_\cap^\circ$ uses sets instead of intersections, writing $\{A, B\} \rightarrow C$ rather than $(A \wedge B) \rightarrow C$. This can be understood only as a difference in notation, given that the intersection type constructor of [12] is assumed to be associative, commutative, and idempotent. Second, $\Lambda_\cap^\circ$ allows sets (*i.e.* intersections) to appear only to the *left* of the arrow ($\rightarrow$) connective. For example, $(A \wedge B) \rightarrow (C \wedge D)$ is a well-formed type in the system of [12], whereas $\{A, B\} \rightarrow \{C, D\}$ is **not** a well-formed type in $\Lambda_\cap^\circ$. This means that types in $\Lambda_\cap^\circ$ are *strict types*, in the sense introduced by van Bakel [31]. Despite these two differences, we will prove that the original system of [12] and $\Lambda_\cap^\circ$ are equivalent in typability power, as they both characterize strongly normalizable λ -terms.

2.2 Intrinsically typed system

We define a Church-style presentation of the system, inspired by the linearization proposed by Kfoury [21].

Definition 2.4 (The $\Lambda_\cap^i$ type assignment system). The sets of **terms** ($t, s, \dots$) and **set-terms** ($\vec{t}, \vec{s}, \dots$) are given by the following grammar:

$$t ::= x^A \mid \lambda x^{\vec{A}}. t \mid t \vec{t} \quad \vec{t} ::= \{t_j\}_{j \in J}$$

where J stands for a finite set of indices and $\{t_j\}_{j \in J}$ is the set $\{t_j \mid j \in J\}$. Note that set-terms are finite sets of terms. When we write a set-term, we keep the invariant that there are no repetitions, *i.e.* that if $h \neq k$ then $t_h \neq t_k$. The $\Lambda_\cap^i$ calculus is defined by means of two forms of judgement:

1. $\Gamma \vdash_i t : A$ meaning that a term t is of type A under the context Γ .
2. $\Gamma \Vdash_i \vec{t} : \vec{A}$ meaning that a set-term $\vec{t}$ is of set-type $\vec{A}$ under the context Γ .

The typing rules are as follows:

$$\frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_i x^B : B} \text{ i-var} \quad \frac{(\Gamma \vdash_i t_j : A_j)_{j \in J} \quad (\forall h, k \in J. h \neq k \implies A_h \neq A_k)}{\Gamma \Vdash_i \{t_j\}_{j \in J} : \{A_j\}_{j \in J}} \text{ i-many}$$

$$\frac{\Gamma, x : \vec{A} \vdash_i t : B}{\Gamma \vdash_i \lambda x^{\vec{A}}. t : \vec{A} \rightarrow B} \text{ i-I} \rightarrow \quad \frac{\Gamma \vdash_i t : \vec{A} \rightarrow B \quad \Gamma \Vdash_i \vec{s} : \vec{A} \quad \vec{A} \neq \emptyset}{\Gamma \vdash_i t \vec{s} : B} \text{ i-E} \rightarrow$$

A term t is **typable** if $\Gamma \vdash_i t : A$ holds for some Γ, A . Similarly, a set-term is typable if $\Gamma \Vdash_i \vec{t} : \vec{A}$ holds for some $\Gamma, \vec{A}$. Unless otherwise specified, when we speak of *term* (resp. *set-term*) we mean *typable term* (resp. *typable set-term*). We write $\Lambda_\cap^i$ for the set of typable terms, i.e. $\Lambda_\cap^i := \{t \mid t \text{ is typable}\}$. The notions of free and bound occurrences of variables are defined as usual. The set of **free variables of type** A of a term t is written $\text{fv}_A(t)$ and defined as the set of variables x such that x^A occurs free in t . For example, $\text{fv}_A(\lambda y^{\{\{A,B\} \rightarrow C\}}. y^{\{A,B\} \rightarrow C} \{x^A, z^B\}) = \{x\}$ when $A \neq B$. The set of **free variables** of a term t is written $\text{fv}(t)$. Terms are considered up to α -renaming of bound variables.

Notation 2.5. Sometimes we write ts to stand for $t\{s\}$. We may omit type annotations over variables if they are clear from the context.

Remark 2.6. Note that, in the rule **i-many**, we explicitly require that the function $j \mapsto A_j$ is injective (i.e. $\forall h, k \in J. h \neq k \implies A_h \neq A_k$). For example, let $\Gamma = (x : \{A, B\}, y : \{A\})$. Then the following are **valid** judgments:

$$\Gamma \Vdash_i \{x^A\} : \{A\} \quad \Gamma \Vdash_i \{x^A, x^B\} : \{A, B\} \quad \Gamma \Vdash_i \{x^B, y^A\} : \{A, B\}$$

On the other hand, the following judgments are **not valid**:

$$\Gamma \Vdash_i \{x^A, y^A\} : \{A\} \quad \Gamma \Vdash_i \{x^A, x^A\} : \{A\}$$

Example 2.7. The derivation of Example 2.2 can be encoded in $\Lambda_\cap^i$ as the following term, which has type $\{\{A, B\} \rightarrow C, A, B\} \rightarrow C$ in the empty context:

$$\lambda x^{\{\{A,B\} \rightarrow C, A, B\}}. x^{\{A,B\} \rightarrow C} \{x^A, x^B\}$$

Remark 2.8. Note that, in general, a set-term may be an *empty* set of terms, but the argument of an application is always a *non-empty* set-term. We let set-types and set-terms to be empty to be able to write expressions such as $\vec{t} = \{t\} \cup \vec{s}$ to mean that $\vec{t}$ is a set containing at least one element t , leaving the possibility that $\vec{s}$ may be empty.

Remark 2.9 (Subterm property). If t is typable, all of its subterms are typable.

Lemma 2.10 (Weakening and strengthening).

1. If $\Gamma \vdash_i t : A$ (resp. $\Gamma \Vdash_i \vec{t} : \vec{A}$) and $\Gamma \subseteq \Delta$ then $\Delta \vdash_i t : A$ (resp. $\Delta \Vdash_i \vec{t} : \vec{A}$).
2. If $\Gamma, x : \{A\} \cup \vec{B} \vdash_i t : C$ (resp. $\Gamma, x : \{A\} \cup \vec{B} \Vdash_i \vec{t} : \vec{C}$) and $x \notin \text{fv}_A(t)$, then $\Gamma \vdash_i t : C$ (resp. $\Gamma \Vdash_i \vec{t} : \vec{C}$), where Γ' is $(\Gamma, x : \vec{B})$ if $\vec{B} \neq \emptyset$, and Γ otherwise.

Definition 2.11 (Minimal typing context). For each term t (resp. set-term $\vec{t}$), its *minimal typing context* is written $\gamma(t)$ (resp. $\gamma(\vec{t})$) and defined as follows:

$$\begin{aligned} \gamma(x^A) &:= x : \{A\} & \gamma(\lambda x^{\vec{A}}. t) &:= \gamma(t) \setminus x \\ \gamma(t \vec{s}) &:= \gamma(t) \cup \gamma(\vec{s}) & \gamma(\{t_1, \dots, t_n\}) &:= \cup_{i=1}^n \gamma(t_i) \end{aligned}$$

where $\Gamma \setminus x$ is defined in such a way that $(\Gamma \setminus x)(x) = \emptyset$ and $(\Gamma \setminus x)(y) = \Gamma(y)$ for $x \neq y$.

Lemma 2.12 (Type uniqueness). If $\Gamma_1 \vdash_i t : A_1$ and $\Gamma_2 \vdash_i t : A_2$ then $A_1 = A_2$. Furthermore, $\gamma(t) \subseteq \Gamma_1$ and $\gamma(t) \subseteq \Gamma_2$ and $\gamma(t) \vdash_i t : A_1$.

Remark 2.13. Sometimes we write t^A to mean that $\gamma(t) \vdash_i t : A$. Note that, by the previous lemma, there is a unique A such that t^A .

Observation 2.14 (Bijection between a set-term and its set-type). Let $\Gamma \Vdash_i \vec{s} : \vec{A}$. Given $A' \in \vec{A}$, we know by rule **i-many** that there exists a derivation $\Gamma \vdash_i s' : A'$. This is the only derivation with type A' in the premises of the rule: otherwise, we would contradict the rule's side condition by having different derivations with the same type.

Therefore, s' is unique. Given $s' \in \vec{s}$, we know by rule **i-many** that it has a type A' ; by Lemma 2.12, this type is unique.

Definition 2.15 (Substitution in $\Lambda_{\cap}^i$). We define operations of capture-avoiding substitution for terms and set-terms by mutual recursion.

Let $\Gamma \Vdash_i \vec{s} : \vec{A}$ with $\vec{s} = \{s_i\}_{i \in I}$, $\vec{A} = \{A_i\}_{i \in I}$, and $\Gamma \vdash_i s_i : A_i$ for all $i \in I$.
we define $t[x^{\vec{A}} := \vec{s}]$ recursively as follows.

$$\begin{aligned} x^{A_i}[x^{\vec{A}} := \vec{s}] &:= s_i \\ y^B[x^{\vec{A}} := \vec{s}] &:= y^B \\ (\lambda y^{\vec{B}}. t)[x^{\vec{A}} := \vec{s}] &:= \lambda y^{\vec{B}}. t[x^{\vec{A}} := \vec{s}] \\ (t \vec{u})[x^{\vec{A}} := \vec{s}] &:= t[x^{\vec{A}} := \vec{s}] \vec{u}[x^{\vec{A}} := \vec{s}] \\ \{t_1, \dots, t_n\}[x^{\vec{A}} := \vec{s}] &:= \{t_1[x^{\vec{A}} := \vec{s}], \dots, t_n[x^{\vec{A}} := \vec{s}]\} \end{aligned}$$

Note that, in the first case, s_i exists and is unique due to Observation 2.14.

Lemma 2.16 (Substitution Lemma). *Let $\Gamma, x : \vec{A} \vdash_i t : B$ and $\Gamma \Vdash_i \vec{s} : \vec{A}$. Then $\Gamma \vdash_i t[x^{\vec{A}} := \vec{s}] : B$.*

Proof. By induction on t , generalizing the statement to set-terms, where the interesting cases are:

1. If $t = x^B$, then $\Gamma, x : \vec{A} \vdash_i x^B : B$ with $B \in \vec{A}$. By Observation 2.14, there exists a unique $s_i \in \vec{s}$ such that $\Gamma \vdash_i s_i : B$. Therefore, $x^B[x^{\vec{A}} := \vec{s}] = s_i$ of type B .
2. If $\vec{t} = \{t_1, \dots, t_n\}$, then B is indeed a set-type $\vec{B} = \{B_1, \dots, B_n\}$, and $\Gamma, x : \vec{A} \vdash_i \{t_1, \dots, t_n\} : \{B_1, \dots, B_n\}$. By IH, $\Gamma \vdash_i t_i[x^{\vec{A}} := \vec{s}] : B_i$ for all i such that $1 \leq i \leq n$, so by **i-many** we have $\Gamma \vdash_i \{t_1[x^{\vec{A}} := \vec{s}], \dots, t_n[x^{\vec{A}} := \vec{s}]\} : \{B_1, \dots, B_n\}$. Note that substitution preserves cardinality in typed set-term derivations. If we had $t_i[x^{\vec{A}} := \vec{s}] = t_j[x^{\vec{A}} := \vec{s}]$ with distinct i, j such that $1 \leq i, j \leq n$, they would have the same type by Lemma 2.12. This would imply that t_i and t_j have the same type, which is impossible by rule **i-many**.

□

Definition 2.17 (Reduction in $\Lambda_{\cap}^i$). A notion of reduction called **i-reduction** is defined over terms and set-terms by the following rule, closed by congruence under arbitrary contexts:

$$(\lambda x^{\vec{A}}. t) \vec{s} \rightarrow_i t[x^{\vec{A}} := \vec{s}]$$

Note in particular that congruence rules allow reducing inside the argument of an application, *i.e.* $\vec{s} \rightarrow_i \vec{u}$ implies $t \vec{s} \rightarrow_i t \vec{u}$, and inside any of the elements of a set-term, *i.e.* $t_i \rightarrow_i t'_i$ implies $\{t_1, \dots, t_i, \dots, t_n\} \rightarrow_i \{t_1, \dots, t'_i, \dots, t_n\}$.

We write $\rightarrow_i^*$ for the reflexive-transitive closure of $\rightarrow_i$.

Reduction in $\Lambda_{\cap}^i$ enjoys the following properties. See Appendices A.1.1 and A.1.2 for detailed proofs.

Proposition 2.18 (Subject reduction). *If $t \rightarrow_i t'$ and $\Gamma \vdash_i t : A$ then $\Gamma \vdash_i t' : A$.*

Proposition 2.19 (Confluence). *If $t \rightarrow_i^* s$ and $t \rightarrow_i^* u$ then there is r such that $s \rightarrow_i^* r$ and $u \rightarrow_i^* r$.*

2.3 Correspondence between Curry and Church-style systems

A typable term in the Church-style system $\Lambda_{\cap}^i$ does not necessarily represent a derivation tree in the Curry-style system $\Lambda_{\cap}^e$. To relate these systems, we consider the following notion:

Definition 2.20 (Refinement). We define a binary relation $t \sqsubset M$ between typable terms $t \in \Lambda_{\cap}^i$ and untyped λ -terms $M \in \Lambda$, as well as a binary relation $\vec{t} \sqsubset M$ between typable set-terms $\vec{t} \in \Lambda_{\cap}^i$ and untyped λ -terms $M \in \Lambda$, according to the inductive rules below. In that case, we say that t is a **refinement** of M .

$$\frac{}{x^A \sqsubset x} \quad \frac{t \sqsubset M}{\lambda x^A. t \sqsubset \lambda x. M} \quad \frac{t \sqsubset M \quad \vec{s} \sqsubset N}{t \vec{s} \sqsubset M N} \quad \frac{(t_i \sqsubset M)_{i \in 1..n} \quad n > 0}{\{t_1, \dots, t_n\} \sqsubset M}$$

Observation 2.21.

1. If $t \sqsubset M_1$ and $t \sqsubset M_2$ then $M_1 = M_2$.
2. If $t \sqsubset M$ and $\vec{s} \sqsubset N$ then $t[x^{\vec{A}} := \vec{s}] \sqsubset M[x := N]$.

Definition 2.22 (Uniformity and type erasure). A typable term $t \in \Lambda_{\cap}^i$ is **uniform** if there exists $M \in \Lambda$ such that $t \sqsubset M$. Similarly, a typable set-term $\vec{t} \sqsubseteq \Lambda_{\cap}^i$ is uniform if there exists $M \in \Lambda$ such that $\vec{t} \sqsubset M$.

If $t \in \Lambda_{\cap}^i$ is a uniform term, its **type erasure** $t^e \in \Lambda$ is defined as the unique λ -term such that $t \sqsubset t^e$. Similarly, if $\vec{t} \sqsubseteq \Lambda_{\cap}^i$ is a uniform set-term, $\vec{t}^e \in \Lambda$ is defined as the unique λ -term such that $\vec{t} \sqsubset \vec{t}^e$. This can be written as a recursive definition:

$$(x^A)^e := x \quad (\lambda x^{\vec{A}}. t)^e := \lambda x. t^e \quad (t \vec{s})^e := t^e \vec{s}^e \quad \{s_1, \dots, s_n\}^e := s_1^e$$

If $t \in \Lambda_{\cap}^i$ is uniform, for each derivation $\Pi \triangleright \Gamma \vdash_i t : A$, we construct a derivation $\Pi^e \triangleright \Gamma \vdash_e t^e : A$ by erasing all the type annotations in terms. Similarly, each derivation $\Pi \triangleright \Gamma \vdash_i \vec{t} : \vec{A}$ is mapped to a derivation $\Pi^e \triangleright \Gamma \vdash_e \vec{t}^e : \vec{A}$.

Definition 2.23 (Type decoration). For each derivation $\Pi \triangleright \Gamma \vdash_e M : A$, we construct a term $t \in \Lambda_{\cap}^i$ and a derivation $\Pi^i \triangleright \Gamma \vdash_i t : A$ inductively as below. Similarly, for each derivation $\Pi \triangleright \Gamma \vdash_e M : \vec{A}$ we construct a set-term $\vec{t} \sqsubseteq \Lambda_{\cap}^i$ and a derivation $\Pi^i \triangleright \Gamma \vdash_i \vec{t} : \vec{A}$.

$$\begin{array}{ccc} \frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_e x : B} \text{ e-var} & \rightsquigarrow & \frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_i x^B : B} \text{ i-var} \\[10pt] \frac{\Gamma, x : \vec{A} \vdash_e M : B}{\Gamma \vdash_e \lambda x. M : \vec{A} \rightarrow B} \text{ e-I} \rightarrow & \rightsquigarrow & \frac{\Gamma, x : \vec{A} \vdash_i t : B}{\Gamma \vdash_i \lambda x^{\vec{A}}. t : \vec{A} \rightarrow B} \text{ i-I} \rightarrow \\[10pt] \frac{\Gamma \vdash_e M : \vec{A} \rightarrow B \quad \Gamma \vdash_e N : \vec{A}}{\Gamma \vdash_e M N : B} \text{ e-E} \rightarrow & \rightsquigarrow & \frac{\Gamma \vdash_i t : \vec{A} \rightarrow B \quad \Gamma \vdash_i \vec{s} : \vec{A}}{\Gamma \vdash_i t \vec{s} : B} \text{ i-E} \rightarrow \\[10pt] \frac{(\Gamma \vdash_e N : A_i)_{i \in 1..n}}{\Gamma \vdash_e N : \{A_1, \dots, A_n\}} \text{ e-many} & \rightsquigarrow & \frac{(\Gamma \vdash_i s_i : A_i)_{i \in 1..n}}{\Gamma \vdash_i \{s_1, \dots, s_n\} : \{A_1, \dots, A_n\}} \text{ i-many} \end{array}$$

Theorem 2.24 (Correspondence).

1. If $\Pi \triangleright \Gamma \vdash_e M : A$, then there exists a uniform $t \in \Lambda_{\cap}^i$ such that $\Pi^i \triangleright \Gamma \vdash_i t : A$ and $t \sqsubset M$. Moreover, $(\Pi^i)^e = \Pi$.
2. If $\Pi \triangleright \Gamma \vdash_i t : A$ and t is uniform then $\Pi^e \triangleright \Gamma \vdash_e t^e : A$. Moreover, $(\Pi^e)^i = \Pi$.

Proof.

1. We generalize the statement to set-terms, *i.e.* if $\Pi \triangleright \Gamma \vdash_e M : \vec{A}$ then there exists a uniform $\vec{t} \sqsubseteq \Lambda_{\cap}^i$ such that $\Pi^i \triangleright \Gamma \vdash_i \vec{t} : \vec{A}$ and $\vec{t} \sqsubset M$. We proceed by induction on the derivation Π . The only not immediate case is when $\Pi \triangleright \Gamma \vdash_e M : \vec{A}$. Then this judgement has been obtained by rule **e-many** with premises $(\Gamma \vdash_e M : A_i)_{i \in 1..n}$; by IH there are derivations proving $\Gamma \vdash_e t_i : A_i$, where $t_i \sqsubset M$. Then, by definition, $\{t_1, \dots, t_n\} \sqsubset M$, and the proof follows by rule **i-many**. The other cases come directly from the definition of type decoration.
2. We generalize the statement to set-terms, *i.e.* if $\Pi \triangleright \Gamma \vdash_i \vec{t} : \vec{A}$ and $\vec{t}$ is uniform then $\Pi^e \triangleright \Gamma \vdash_e \vec{t}^e : \vec{A}$. We proceed by induction on the derivation Π . The only not immediate case is when $\Pi \triangleright \Gamma \vdash_i \vec{t} : \vec{A}$. Let $\vec{t} = \{t_1, \dots, t_n\}$, then $t \sqsubset M$ implies, by definition of uniformity, $t_i \sqsubset M$. The judgement has been obtained by rule **i-many**, with premises $\Gamma \vdash_i t_i : A_i$, so by IH, $\Gamma \vdash_e t_i^e : A_i$ where $t_i \sqsubset t_i^e$. By Observation 2.21.1, $t_i \sqsubset t_i^e$ and $t_i \sqsubset M$ imply $t_i^e = M$, so $\Gamma \vdash_e t : A_i$ and the result follows by applying rule **e-many**. The other cases come directly from the definition of type erasure. $\square$

Corollary 2.25. *Given a term $M \in \Lambda$, the following are equivalent:*

1. *There exist Γ, A such that $\Gamma \vdash_{\bullet} M : A$ holds.*
2. *There exists a term $t \in \Lambda_{\cap}^i$ such that $t \sqsubset M$.*

Furthermore, $\rightarrow_{\beta}$ and $\rightarrow_i$ simulate each other:

Theorem 2.26 (Simulation).

1. *If $M \rightarrow_{\beta} N$ and $t \sqsubset M$, there exists $s \in \Lambda_{\cap}^i$ such that $t \rightarrow_i^+ s$ and $s \sqsubset N$.*
2. *If $t \rightarrow_i s$ and $t \sqsubset M$, there exist $N \in \Lambda$ and $s' \in \Lambda_{\cap}^i$ such that $M \rightarrow_{\beta} N$ and $s \rightarrow_i^* s'$ and $s' \sqsubset N$.*

Graphically:

$$\begin{array}{ccc}
 M & \xrightarrow{\beta} & N \\
 \sqsubset & & \sqsubset \\
 t & \xrightarrow{i}^+ & s
 \end{array}
 \quad
 \begin{array}{ccc}
 t & \xrightarrow{i} & s \xrightarrow{i}^* s' \\
 \sqsubset & & \sqsubset \\
 M & \xrightarrow{\beta} & N
 \end{array}$$

Proof.

1. We proceed by induction on M . The case $M = x$ cannot happen, in case $M = \lambda x. N$ the proof follows by IH. Let $M = PQ$, so $t = u\vec{r}$, where $u \sqsubset P$ and $\vec{r} \sqsubset Q$. The case $P \rightarrow_{\beta} P'$ is easy, by IH. Let $Q \rightarrow_{\beta} Q'$, and let $\vec{r} = \{r_1, \dots, r_n\}$, where, by definition, $r_i \sqsubset Q$, for all $1 \leq i \leq n$. By IH, there are r'_i such that $r_i \rightarrow_i^+ r'_i$, such that $r'_i \sqsubset Q'$. So $\{r'_1, \dots, r'_n\} \sqsubset Q'$ and $s = u\{r'_1, \dots, r'_n\}$. Let $P = \lambda x. P'$, so $t = (\lambda x^A. t')\vec{r}$, where $t' \sqsubset P'$, and let $M \rightarrow_{\beta} P'[x := Q]$. By Observation 2.21.2, $t'[x^A := \vec{r}] \sqsubset P'[x := Q]$, and since $t \rightarrow_i^+ t'[x^A := \vec{r}]$, $s = t'[x^A := \vec{r}]$.
2. We generalize the statement to set-terms, *i.e.* if $\vec{t} \rightarrow_i \vec{s}$ and $\vec{t} \sqsubset M$, there exist N and $\vec{s}'$ such that $M \rightarrow_{\beta} N$ and $\vec{s} \rightarrow_i^* \vec{s}'$ and $\vec{s}' \sqsubset N$. The proof is by induction on terms. Let $\vec{t} = \{t_1, \dots, t_n, u\}$, and $\vec{s} = \{s_1, \dots, s_n, u'\}$, where $u \rightarrow_i u'$. By IH, there are N and u'' such that $M \rightarrow_{\beta} N$, $u' \rightarrow_i^* u''$ and $u'' \sqsubset N$. Note that $t_i \sqsubset M$ for all $i \in 1..n$. Now let $i \in 1..n$. By point 1, $M \rightarrow_{\beta} N$ and $t_i \sqsubset M$ imply there is t'_i such that $t_i \rightarrow_i^* t'_i$ and $t'_i \sqsubset N$; then it suffices to take $\vec{s}' = \{t'_1, \dots, t'_n, u''\}$. The other cases follow by IH, using Observation 2.21.2 in case $t = (\lambda x^A. t') \vec{u}$, and $s = t'[x^A := \vec{u}]$. $\square$

Some comments are in order. If N is a redex subterm of M , then a derivation tree for M in $\Lambda_{\cap}^e$ may contain more than one subderivation with subject N . Contracting N with usual β -reduction corresponds to reducing in parallel *all* the occurrences of N in all the subderivations. In $\Lambda_{\cap}^i$, this corresponds instead to performing the reductions one at a time.

Example 2.27. Let $M = (\lambda x. xx)(II)$, $A = B \rightarrow B$, and $I = \lambda x. x$. Consider the derivation Π :

$$\frac{\frac{\vdots}{x : \{A \rightarrow A, A\} \vdash_{\bullet} xx : A} \vdash_{\bullet} \lambda x. xx : \{A \rightarrow A, A\} \rightarrow A}{\vdash_{\bullet} M : A} \text{e-E} \rightarrow \frac{\frac{\vdots}{\vdash_{\bullet} II : A \rightarrow A} \quad \frac{\vdots}{\vdash_{\bullet} II : A}}{\vdash_{\bullet} II : \{A \rightarrow A, A\}} \text{e-many} \text{e-E} \rightarrow$$

Then we have that $\Pi^i \triangleright \vdash_i t : A$, where $t = (\lambda x^{\{A \rightarrow A, A\}}. xx) \{(II)^{A \rightarrow A}, (II)^A\}$. Note that:

1. t is a uniform term, and in particular $t \sqsubset M$.
2. $t \rightarrow_i t_1 = (\lambda x^{\{A \rightarrow A, A\}}. xx) \{I^{A \rightarrow A}, (II)^A\}$, where t_1 is not a uniform term.
3. $t \rightarrow_i t_3 = (II)^{A \rightarrow A} (II)^A$ where $t_3 \sqsubset II(II)$ and $M \rightarrow_{\beta} II(II)$.
4. $t_1 \rightarrow_i^* t_2 = (\lambda x^{\{A \rightarrow A, A\}}. xx) \{I^{A \rightarrow A}, I^A\}$, where t_2 is a uniform term; in particular $t_2 \sqsubset (\lambda x. xx)I$ and $M \rightarrow_{\beta} (\lambda x. xx)I$.

Lemma 2.28 (Strong Normalization typability). *Let $M \in \Lambda$ be strongly normalizing. Then there is $t \in \Lambda_{\cap}^i$ such that $t \sqsubset M$ and $\Gamma \vdash_i t : A$, for some Γ and A .*

Proof. Following the inductive definition of strongly normalizing terms, using the notion of refinement and a head subject expansion lemma. See Appendix A.1.3 for the detailed proof. $\square$

3 Strong normalization via a decreasing measure

In this section, we prove strong normalization of the $\Lambda_{\cap}^i$ calculus by providing an explicit decreasing measure, adapting the ideas behind the $\mathcal{W}$ measure of [2] to the setting of idempotent intersection type systems. The $\mathcal{W}$ measure is based on the fact that contracting a redex in the simply typed λ -calculus cannot create a redex of higher or equal *degree*, where the degree of a redex is defined as the height of the type of its abstraction. This observation was already known to Turing, as reported by Gandy [19].

This section is organized as follows. In Section 3.1, we enrich the syntax of $\Lambda_{\cap}^i$ by defining an auxiliary *memory* calculus $\Lambda_{\cap}^{\text{im}}$ which incorporates *wrappers* $\langle \vec{s} \rangle$, and we study some technical properties that are needed later. In Section 3.2 we define an operation called *full simplification* for $\Lambda_{\cap}^{\text{im}}$ terms, which iteratively contracts in parallel all redexes of maximum degree, showing that this yields the normal form of the term. Finally, in Section 3.3 we show that $\Lambda_{\cap}^i$ is SN by defining a decreasing measure $\mathcal{W}$, which works by fully simplifying a term in $\Lambda_{\cap}^{\text{im}}$ and counting the number of remaining wrappers.

3.1 The memory calculus $\Lambda_{\cap}^{\text{im}}$

Definition 3.1 (The $\Lambda_{\cap}^{\text{im}}$ calculus). The sets of terms $(t, s, \dots)$ and set-terms $(\vec{t}, \vec{s}, \dots)$ are given by the following grammar:

$$t ::= x^A \mid \lambda x^{\vec{A}}. t \mid t \vec{t} \mid t \langle \vec{t} \rangle \quad \vec{t} ::= \{t_j\}_{j \in J}$$

where J stands for a finite set of indexes. We write $\mathbf{L}$ for a list of wrappers, defined by the grammar $\mathbf{L} ::= \square \mid \mathbf{L} \langle \vec{t} \rangle$, and $t\mathbf{L}$ for the term that results from extending t with all the wrappers in the list, *i.e.* $t(\square \langle \vec{s}_1 \rangle \dots \langle \vec{s}_n \rangle) = t \langle \vec{s}_1 \rangle \dots \langle \vec{s}_n \rangle$. Typing judgements are of the forms $\Gamma \vdash_{\text{im}} t : A$ and $\Gamma \Vdash_{\text{im}} \vec{t} : \vec{A}$, where types and typing contexts are defined as before. The typing rules extend the $\Lambda_{\cap}^i$ type assignment system of Definition 2.4 with a typing rule **im-wrap**:

$$\begin{array}{c} \frac{B \in \vec{A}}{\Gamma, x : \vec{A} \vdash_{\text{im}} x^B : B} \text{im-var} \quad \frac{\Gamma, x : \vec{A} \vdash_{\text{im}} t : B}{\Gamma \vdash_{\text{im}} \lambda x^{\vec{A}}. t : \vec{A} \rightarrow B} \text{im-I} \rightarrow \\[10pt] \frac{\Gamma \vdash_{\text{im}} t : \vec{A} \rightarrow B \quad \Gamma \Vdash_{\text{im}} \vec{s} : \vec{A} \quad \vec{A} \neq \emptyset}{\Gamma \vdash_{\text{im}} t \vec{s} : B} \text{im-E} \rightarrow \quad \frac{\Gamma \vdash_{\text{im}} t : A \quad \Gamma \Vdash_{\text{im}} \vec{s} : \vec{B}}{\Gamma \vdash_{\text{im}} t \langle \vec{s} \rangle : A} \text{im-wrap} \\[10pt] \frac{(\Gamma \vdash_{\text{im}} t_j : A_j)_{j \in J} \quad (\forall h, k \in J. h \neq k \implies A_h \neq A_k)}{\Gamma \Vdash_{\text{im}} \{t_j\}_{j \in J} : \{A_j\}_{j \in J}} \text{im-many} \end{array}$$

The operation of capture-avoiding substitution $t[x^{\vec{A}} := \vec{s}]$ is extended by declaring that $(t \langle \vec{u} \rangle)[x^{\vec{A}} := \vec{s}] = t[x^{\vec{A}} := \vec{s}] \langle \vec{u}[x^{\vec{A}} := \vec{s}] \rangle$. Reduction in the $\Lambda_{\cap}^{\text{im}}$ -calculus, called **im-reduction**, is defined over terms and set-terms by the following rule, closed by congruence under arbitrary contexts:

$$(\lambda x^{\vec{A}}. t)\mathbf{L} \vec{s} \rightarrow_{\text{im}} t[x := \vec{s}] \langle \vec{s} \rangle \mathbf{L}$$

Abstractions followed by lists of wrappers, *i.e.* terms of the form $(\lambda x. t)\mathbf{L}$ are called wrapped abstractions or **w-abstractions** for short. A **redex** is an expression matching the left-hand side of the **im** rule, which must

be an applied w-abstraction. The **height** of a type A (resp. set-type $\vec{A}$) is written $h(A)$ (resp. $h(\vec{A})$) and defined as follows:

$$\begin{aligned} h(a) &:= 0 \\ h(\vec{A} \rightarrow B) &:= 1 + \max\{h(\vec{A}), h(B)\} \\ h(\{A_1, \dots, A_n\}) &:= \max\{h(A_1), \dots, h(A_n)\} \end{aligned}$$

We write $\text{type}(t)$ for the type of t , which is uniquely defined for typable terms. The **degree** of a w-abstraction is the height of its type. The degree of a redex is the degree of its w-abstraction. The **max-degree** of a term $t \in \Lambda_{\cap}^{\text{im}}$ is written $\delta_{\max}(t)$ and defined as the maximum degree of the redexes in t , or 0 if t has no redexes. This notion is also defined for set-terms ($\delta_{\max}(\vec{t})$) and lists of wrappers ($\delta_{\max}(\mathbf{L})$) in a similar way. The **weight** of a term $t \in \Lambda_{\cap}^{\text{im}}$ is written $w(t)$ and defined as the number of wrappers in t .

Remark 3.2. Each step $t \rightarrow_i s$ in the $\Lambda_{\cap}^i$ -calculus (without wrappers) has a **corresponding** step $t \rightarrow_{\text{im}} s'$ in the $\Lambda_{\cap}^{\text{im}}$ -calculus (with wrappers), contracting the same redex but creating one wrapper. For example, the step $(\lambda x. z x x) I \rightarrow_i z I I$ has a corresponding step $(\lambda x. z x x) I \rightarrow_{\text{im}} (z I I) \langle I \rangle$. In particular an erasing reduction step in $\Lambda_{\cap}^i$ is mapped a non-erasing one in $\Lambda_{\cap}^{\text{im}}$. For example, $(\lambda x^{\vec{A}}. z^B) y^{\vec{A}} \rightarrow_i z^B$, while $(\lambda x^{\vec{A}}. z^B) y^{\vec{A}} \rightarrow_{\text{im}} z^B \langle y^{\vec{A}} \rangle$.

The following properties hold for the $\Lambda_{\cap}^{\text{im}}$ calculus. Subject expansion also holds for this calculus, but it is not needed as part of the technical development. See Appendices A.2.1 and A.2.2 for detailed proofs.

Proposition 3.3 (Subject reduction). *If $t \rightarrow_{\text{im}} t'$ and $\Gamma \vdash_{\text{im}} t : A$, then $\Gamma \vdash_{\text{im}} t' : A$.*

Proposition 3.4 (Confluence). *If $t_1 \rightarrow_{\text{im}}^* t_2$ and $t_1 \rightarrow_{\text{im}}^* t_3$, there exists a term t_4 such that $t_2 \rightarrow_{\text{im}}^* t_4$ and $t_3 \rightarrow_{\text{im}}^* t_4$.*

3.2 Simplification by complete developments

Definition 3.5 (Simplification). For each integer $d \geq 1$ we define an operation written $S_d(-)$, called **simplification of degree d** , that can be applied on terms ($S_d(t)$), set-terms ($S_d(\vec{t})$), and lists of wrappers ($S_d(\mathbf{L})$), mutually recursively as follows:

$$\begin{aligned} S_d(x^A) &:= x^A \\ S_d(\lambda x^{\vec{A}}. t) &:= \lambda x^{\vec{A}}. S_d(t) \\ S_d(t \vec{s}) &:= \begin{cases} S_d(t') [x^{\vec{A}} := S_d(\vec{s})] \langle S_d(\vec{s}) \rangle S_d(\mathbf{L}) & \text{if } t = (\lambda x^{\vec{A}}. t') \mathbf{L} \text{ of degree } d \\ S_d(t) S_d(\vec{s}) & \text{otherwise} \end{cases} \\ S_d(t \langle \vec{s} \rangle) &:= S_d(t) \langle S_d(\vec{s}) \rangle \\ S_d(\{t_1, \dots, t_n\}) &:= \{S_d(t_1), \dots, S_d(t_n)\} \\ S_d(\square \langle \vec{t}_1 \rangle \dots \langle \vec{t}_n \rangle) &:= \square \langle S_d(\vec{t}_1) \rangle \dots \langle S_d(\vec{t}_n) \rangle \end{aligned}$$

If $t \in \Lambda_{\cap}^{\text{im}}$ is a term and $D = \delta_{\max}(t)$, the **full simplification of t** is written $S_*(t)$ and defined as $S_*(t) := S_1(\dots S_{D-1}(S_D(t)) \dots)$.

Remark 3.6. Simplification $S_d(t)$ computes the result of the *complete development* of the redexes of degree d in t .

Lemma 3.7 (Soundness of simplification). *If $t \in \Lambda_{\cap}^{\text{im}}$ and $d \geq 1$ then $t \rightarrow_{\text{im}}^* S_d(t)$.*

Proof. Straightforward by mutual induction on the term t , set-term $\vec{t}$ and list of wrappers $\mathbf{L}$, generalizing the statement to set-terms and lists of wrappers, i.e. $\vec{t} \rightarrow_{\text{im}}^* S_d(\vec{t})$ and $\mathbf{L} \rightarrow_{\text{im}}^* S_d(\mathbf{L})$. $\square$

Lemma 3.8 (Creation of abstraction by substitution). *Let $t \in \Lambda_{\cap}^{\text{im}}$ and $\vec{s} \subset \Lambda_{\cap}^{\text{im}}$ such that t is not a w-abstraction but $t[x^{\vec{A}} := \vec{s}]$ is a w-abstraction. Then $t = x^B \mathbf{L}$ with $B \in \vec{A}$.*

Proof. Let us write t as of the form $t = t'L$ where t' is not a wrapper. Note that t' is not an abstraction, because by hypothesis t is not a w-abstraction. Moreover, t' cannot be an application, because if $t' = t_1 t_2$ then $t[x^{\vec{A}} := \vec{s}] = t'[x^{\vec{A}} := \vec{s}](L[x^{\vec{A}} := \vec{s}]) = (t_1[x^{\vec{A}} := \vec{s}] t_2[x^{\vec{A}} := \vec{s}])(L[x^{\vec{A}} := \vec{s}])$ would not be a w-abstraction. The only remaining possibility is that t' is a variable, so $t' = y^B$. If it is a variable other than x , i.e. $y \neq x$, then $t[x^{\vec{A}} := \vec{s}] = t'[x^{\vec{A}} := \vec{s}](L[x^{\vec{A}} := \vec{s}]) = y^B(L[x^{\vec{A}} := \vec{s}])$ would not be a w-abstraction. So we necessarily have that $y = x^B$ with $B \in \vec{A}$, which concludes the proof. $\square$

Lemma 3.9 (Bound for the max-degree of a substitution). *Let $\Gamma \Vdash_{\text{im}} \vec{s} : \vec{A}$, where $\delta_{\max}(\vec{s}) < d$ and $\mathbf{h}(\vec{A}) < d$. If $\Gamma, x : \vec{A} \vdash_{\text{im}} t : B$ and $\delta_{\max}(t) < d$ then $\delta_{\max}(t[x^{\vec{A}} := \vec{s}]) < d$.*

Proof. We generalize the statement for set-terms, claiming that $\Gamma, x : \vec{A} \vdash_{\text{im}} \vec{t} : \vec{B}$ and $\delta_{\max}(\vec{t}) < d$ imply $\delta_{\max}(\vec{t}[x^{\vec{A}} := \vec{s}]) < d$. The proof proceeds by simultaneous induction on the structure of the term t or set-term $\vec{t}$. The interesting cases are when the term is a variable or an application. The remaining cases are straightforward by resorting to the IH. If t is a **variable**, $t = y^C$, we consider two subcases, depending on whether $y = x$ or not.

1. If $x = y$, then $C = B$ and since $\Gamma, x : \vec{A} \vdash_{\text{im}} x : B$ holds, we have that $B \in \vec{A}$. Hence there is a term $s_0 \in \vec{s}$ such that $\Gamma \vdash_{\text{im}} s_0 : B$, and $x^B[x^{\vec{A}} := \vec{s}] = s_0$. In particular, by definition we have that $\delta_{\max}(s_0) \leq \delta_{\max}(\vec{s})$, and by hypothesis we have that $\delta_{\max}(\vec{s}) < d$, so $\delta_{\max}(t[x^{\vec{A}} := \vec{s}]) = \delta_{\max}(x^B[x^{\vec{A}} := \vec{s}]) = \delta_{\max}(s_0) < d$, as required.
2. If $x \neq y$, then $\delta_{\max}(t[x^{\vec{A}} := \vec{s}]) = \delta_{\max}(y^C[x^{\vec{A}} := \vec{s}]) = \delta_{\max}(y^C) = \delta_{\max}(t) < d$.

If t is an **application**, $t = u \vec{r}$, note that u is a subterm of t so $\delta_{\max}(u) \leq \delta_{\max}(t) < d$, and similarly $\delta_{\max}(\vec{r}) \leq \delta_{\max}(t) < d$. Thus applying the IH on each subterm we have that $\delta_{\max}(u[x^{\vec{A}} := \vec{s}]) < d$ and that $\delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}]) < d$. We consider three subcases, depending on whether **1.** u is a w-abstraction, **2.** u is not a w-abstraction and $u[x^{\vec{A}} := \vec{s}]$ is a w-abstraction, or **3.** u and $u[x^{\vec{A}} := \vec{s}]$ are not w-abstractions:

1. If u is a w-abstraction: then $u = (\lambda y^{\vec{C}}. u')L$ is of type $\vec{C} \rightarrow D$. Let k be the degree of the w-abstraction u , i.e. the height of its type, $k = \mathbf{h}(\vec{C} \rightarrow D)$. By definition, $\delta_{\max}(t) = \delta_{\max}(u \vec{r}) = \max\{k, \delta_{\max}(u), \delta_{\max}(\vec{r})\}$. Since $\delta_{\max}(t) < d$ by hypothesis, in particular we have that $k < d$. Moreover, note that $u[x^{\vec{A}} := \vec{s}] = (\lambda y^{\vec{C}}. u'[x^{\vec{A}} := \vec{s}])(L[x^{\vec{A}} := \vec{s}])$, so $u[x^{\vec{A}} := \vec{s}]$ is a w-abstraction, and it is of degree k because substitution preserves the type of a term. Therefore:

$$\begin{aligned} \delta_{\max}(t[x^{\vec{A}} := \vec{s}]) &= \delta_{\max}(u[x^{\vec{A}} := \vec{s}] \vec{r}[x^{\vec{A}} := \vec{s}]) \\ &= \max\{k, \delta_{\max}(u[x^{\vec{A}} := \vec{s}]), \delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}])\} < d \end{aligned}$$

The last step is justified because we have already noted that $k < d$ and $\delta_{\max}(u[x^{\vec{A}} := \vec{s}]) < d$ and $\delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}]) < d$.

2. If u is not a w-abstraction and $u[x^{\vec{A}} := \vec{s}]$ is a w-abstraction: then by Lemma 3.8 u must be of the form $x^C L$ with $A \in C$. Let k be the degree of the w-abstraction $u[x^{\vec{A}} := \vec{s}]$. Then k is the height of the type of $u[x^{\vec{A}} := \vec{s}]$, that is, $k = \mathbf{h}(C) \leq \mathbf{h}(\vec{A}) < d$. To conclude, note that:

$$\begin{aligned} \delta_{\max}(t[x^{\vec{A}} := \vec{s}]) &= \delta_{\max}(u[x^{\vec{A}} := \vec{s}] \vec{r}[x^{\vec{A}} := \vec{s}]) \\ &= \max\{k, \delta_{\max}(u[x^{\vec{A}} := \vec{s}]), \delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}])\} < d \end{aligned}$$

The last step is justified because we have already noted that $k < d$ and $\delta_{\max}(u[x^{\vec{A}} := \vec{s}]) < d$ and $\delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}]) < d$.

3. If u and $u[x^{\vec{A}} := \vec{s}]$ are not a w-abstractions: then

$$\begin{aligned}\delta_{\max}(t[x^{\vec{A}} := \vec{s}]) &= \delta_{\max}(u[x^{\vec{A}} := \vec{s}] \vec{r}[x^{\vec{A}} := \vec{s}]) \\ &= \max\{\delta_{\max}(u[x^{\vec{A}} := \vec{s}]), \delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}])\} < d\end{aligned}$$

The last step is justified because we have already noted that $\delta_{\max}(u[x^{\vec{A}} := \vec{s}]) < d$ and $\delta_{\max}(\vec{r}[x^{\vec{A}} := \vec{s}]) < d$. $\square$

Lemma 3.10 (Simplification does not create abstractions). *Suppose that $t \in \Lambda_{\cap}^{\text{im}}$ is not a w-abstraction and $\delta_{\max}(t) \leq d$ and $\mathbf{h}(\text{type}(t)) \geq d$. Then $\mathbf{S}_d(t)$ is not a w-abstraction.*

Proof. We proceed by induction on t . The interesting case is when t is a redex of degree d . We claim that this case is impossible. Indeed, suppose that $t = (\lambda x^{\vec{A}}. s) \mathbf{L} \vec{u}$, where $\lambda x^{\vec{A}}. s$ is of type $\vec{A} \rightarrow B$ so that t is of type B . The degree of the redex is $d = \mathbf{h}(\vec{A} \rightarrow B)$ and $\mathbf{h}(\text{type}(t)) = \mathbf{h}(B) < d$. However, by hypothesis, $\mathbf{h}(\text{type}(t)) \geq d$, from which we obtain a contradiction. The remaining cases are straightforward resorting to the IH. $\square$

Lemma 3.11 (Simplification decreases the max-degree). *If $d \geq 1$ and $\delta_{\max}(t) \leq d$ then $\delta_{\max}(\mathbf{S}_d(t)) < d$.*

Proof. We proceed by simultaneous induction, generalizing the statement for set-terms ($\delta_{\max}(\vec{t}) \leq d$ implies $\delta_{\max}(\mathbf{S}_d(\vec{t})) < d$) and lists of wrappers ($\delta_{\max}(\mathbf{L}) \leq d$ implies $\delta_{\max}(\mathbf{S}_d(\mathbf{L})) < d$). The interesting cases are when t is a variable or an application. The remaining cases are straightforward by IH. If t is a **variable**, $t = x^{\vec{A}}$, then $\delta_{\max}(\mathbf{S}_d(x^{\vec{A}})) = \delta_{\max}(x^{\vec{A}}) = 0 < d$ because $d \geq 1$ by hypothesis. If t is an **application**, $t = s \vec{u}$, we consider two cases, depending on whether s is a w-abstraction of degree d or not:

1. If $s = (\lambda x^{\vec{A}}. s') \mathbf{L}$ is a w-abstraction of degree d : then note that s' is a subterm of t which implies that $\delta_{\max}(s') \leq \delta_{\max}(t) \leq d$, and this in turn implies by IH that $\delta_{\max}(\mathbf{S}_d(s')) < d$. Similarly, we can note that $\delta_{\max}(\mathbf{L}) \leq d$ so by IH $\delta_{\max}(\mathbf{S}_d(\mathbf{L})) < d$, and that $\delta_{\max}(\vec{u}) \leq d$ so by IH $\delta_{\max}(\mathbf{S}_d(\vec{u})) < d$. Since the term is well-typed, the type of the w-abstraction $s = (\lambda x^{\vec{A}}. s') \mathbf{L}$ is of the form $\vec{A} \rightarrow B$, while the type of $\vec{u}$ is $\vec{A}$. Moreover, $\mathbf{h}(\vec{A} \rightarrow B) = d$ by hypothesis, so in particular $\mathbf{h}(\vec{A}) < d$. From this we obtain by Lemma 3.9 that $\delta_{\max}(\mathbf{S}_d(s'))[x^{\vec{A}} := \mathbf{S}_d(\vec{u})] < d$. Finally, we have:

$$\begin{aligned}\delta_{\max}(\mathbf{S}_d(s \vec{u})) &= \delta_{\max}(\mathbf{S}_d(s') [x^{\vec{A}} := \mathbf{S}_d(\vec{u})] \langle \mathbf{S}_d(\vec{u}) \rangle \mathbf{S}_d(\mathbf{L})) \\ &= \max\{\delta_{\max}(\mathbf{S}_d(s') [x^{\vec{A}} := \mathbf{S}_d(\vec{u})]), \delta_{\max}(\mathbf{S}_d(\vec{u})), \delta_{\max}(\mathbf{S}_d(\mathbf{L}))\} < d\end{aligned}$$

2. If s is not a w-abstraction of degree d : since s is a subterm of t we have that $\delta_{\max}(s) \leq \delta_{\max}(t) \leq d$ so by IH $\delta_{\max}(\mathbf{S}_d(s)) < d$. Similarly, $\delta_{\max}(\vec{u}) \leq \delta_{\max}(t) \leq d$ so by IH $\delta_{\max}(\mathbf{S}_d(\vec{u})) < d$. Let the type of s be of the form $\vec{A} \rightarrow B$ and let $k := \mathbf{h}(\vec{A} \rightarrow B)$. We consider two cases, depending on whether $k < d$ or $k \geq d$.

If $k = \mathbf{h}(\vec{A} \rightarrow B) < d$: then by Lemma 3.7 we have $s \rightarrow_{\text{im}}^* \mathbf{S}_d(s)$ and by Subject Reduction (3.3) the type of $\mathbf{S}_d(s)$ is also $\vec{A} \rightarrow B$. Hence:

$$\delta_{\max}(\mathbf{S}_d(t)) = \delta_{\max}(\mathbf{S}_d(s \vec{u})) = \delta_{\max}(\mathbf{S}_d(s) \mathbf{S}_d(\vec{u})) = \max\{k, \delta_{\max}(s), \delta_{\max}(\vec{u})\} < d$$

On the other hand, if $k = \mathbf{h}(\vec{A} \rightarrow B) \geq d$, we claim that s cannot be a w-abstraction. First, note by hypothesis that s is not a w-abstraction of degree $k = d$. Second, s cannot be a w-abstraction of degree $k > d$, as we would have that $s \vec{u}$ is a redex of degree k , so $\delta_{\max}(s \vec{u}) \geq k$. This would imply that $d < k \leq \delta_{\max}(s \vec{u}) \leq d$, a contradiction. Since s is not a w-abstraction, its type is $\vec{A} \rightarrow B$ of height $k \geq d$, and $\delta_{\max}(s) \leq d$, by Lemma 3.10 we have that $\mathbf{S}_d(s)$ is not a w-abstraction. Hence $\mathbf{S}_d(s) \mathbf{S}_d(\vec{u})$ is not a redex, and $\delta_{\max}(\mathbf{S}_d(t)) = \delta_{\max}(\mathbf{S}_d(s \vec{u})) = \delta_{\max}(\mathbf{S}_d(s) \mathbf{S}_d(\vec{u})) = \max\{\delta_{\max}(\mathbf{S}_d(s)), \delta_{\max}(\mathbf{S}_d(\vec{u}))\} < d$. $\square$

Proposition 3.12 (Full simplification yields the normal form). *Let $t \in \Lambda_{\cap}^{\text{im}}$. Then $t \rightarrow_{\text{im}}^* \mathbf{S}_*(t)$ and $\mathbf{S}_*(t)$ is in $\rightarrow_{\text{im}}$ -normal form.*

Proof. Let d be the max-degree of t and define $\mathbf{S}_{>k}(t) := \mathbf{S}_{k+1}(\dots \mathbf{S}_{d-1}(\mathbf{S}_d(t)) \dots)$ for each k such that $0 \leq k \leq d$. Note that $\mathbf{S}_{>d}(t) = t$ and $\mathbf{S}_{>0}(t) = \mathbf{S}_*(t)$. To show that $t \rightarrow_{\text{im}}^* \mathbf{S}_*(t)$, note that $\mathbf{S}_{>k}(t) \rightarrow_{\text{im}}^* \mathbf{S}_k(\mathbf{S}_{>k}(t)) = \mathbf{S}_{>k-1}(t)$ for each $1 \leq k \leq d$ by Lemma 3.7, so:

$$t = \mathbf{S}_{>d}(t) \rightarrow_{\text{im}}^* \mathbf{S}_{>d-1}(t) \rightarrow_{\text{im}}^* \dots \rightarrow_{\text{im}}^* \mathbf{S}_{>k}(t) \rightarrow_{\text{im}}^* \dots \rightarrow_{\text{im}}^* \mathbf{S}_{>0}(t) = \mathbf{S}_*(t)$$

To show that $\mathbf{S}_*(t)$ is a $\rightarrow_{\text{im}}$ -normal form, we claim that for each $0 \leq k \leq d$ we have that $\delta_{\max}(\mathbf{S}_{>k}(t)) \leq k$. We proceed by induction on $d - k$. If $d - k = 0$, we have that $k = d$, so $\delta_{\max}(\mathbf{S}_{>d}(t)) = \delta_{\max}(t) = d$, since d is the max-degree of t . If $d - k > 0$, we have that $0 \leq k < d$. By IH, $\delta_{\max}(\mathbf{S}_{>k+1}(t)) \leq k + 1$. Then $\delta_{\max}(\mathbf{S}_{>k}(t)) = \delta_{\max}(\mathbf{S}_{k+1}(\mathbf{S}_{>k+1}(t))) < k + 1$ by Lemma 3.11. This means that $\delta_{\max}(\mathbf{S}_{>k}(t)) \leq k$, as required. $\square$

3.3 The decreasing measure

Definition 3.13 (Forgetful reduction). We define a binary relation $t \triangleright s$ between typable terms as the closure by congruence under arbitrary contexts of the axiom $t \langle \vec{s} \rangle \triangleright t$.

Remark 3.14. If t is in $\rightarrow_{\text{im}}$ -normal form and $t \triangleright s$ then s is also in $\rightarrow_{\text{im}}$ -normal form.

Remark 3.15. If $t \triangleright s$ then $w(t) > w(s)$.

Lemma 3.16 (Reduce/forget lemma). *Let $t \rightarrow_i s$ be a step in the $\Lambda_{\cap}^i$ -calculus (without wrappers) and consider its corresponding step $t \rightarrow_{\text{im}} s'$. Then $s' \triangleright s$.*

Proof. Straightforward by induction on t . $\square$

Lemma 3.17 (Forgetful reduction commutes with reduction). *If $t_1 \rightarrow_{\text{im}}^* t_2$ and $t_1 \triangleright^+ t_3$ there exists t_4 such that $t_2 \triangleright^+ t_4$ and $t_3 \rightarrow_{\text{im}}^* t_4$.*

Proof. It suffices to prove a local commutation result, namely that if $t_1 \rightarrow_{\text{im}} t_2$ and $t_1 \triangleright t_3$ there exists t_4 such that $t_2 \triangleright^+ t_4$ and $t_3 \rightarrow_{\text{im}}^* t_4$.

We proceed by induction on t_1 . The three interesting cases are shown below.

See Appendix A.2.3 for the complete proof. $\square$

$$\begin{array}{ccccc} (\lambda x^{\vec{A}}. s) \mathbf{L} \vec{u} & \rightarrow_{\text{im}} & s[x^{\vec{A}} := \vec{u}] \langle \vec{u} \rangle \mathbf{L} & s \langle \vec{u} \rangle & \rightarrow_{\text{im}} & s' \langle \vec{u} \rangle & s \langle \vec{u} \rangle & \rightarrow_{\text{im}} & s \langle \vec{u}' \rangle \\ \nabla & & \nabla^+ & \nabla & & \nabla & \nabla & & \nabla \\ (\lambda x^{\vec{A}}. s) \mathbf{L} \vec{u}' & \rightarrow_{\text{im}} & s[x^{\vec{A}} := \vec{u}'] \langle \vec{u}' \rangle \mathbf{L} & s & \rightarrow_{\text{im}} & s' & s & = & s \end{array}$$

Lemma 3.18 (Properties of the full simplification).

1. *If $t \rightarrow_{\text{im}} s$ then $\mathbf{S}_*(t) = \mathbf{S}_*(s)$.* **2.** *If $t \triangleright s$ then $\mathbf{S}_*(t) \triangleright^+ \mathbf{S}_*(s)$.*

Proof. We prove each item separately.

First, suppose that $t \rightarrow_{\text{im}} s$. By the fact that full simplification yields the normal form of a term (Proposition 3.12), we know that $t \rightarrow_{\text{im}}^* \mathbf{S}_*(t)$ and $s \rightarrow_{\text{im}}^* \mathbf{S}_*(s)$, where both $\mathbf{S}_*(t)$ and $\mathbf{S}_*(s)$ are in $\rightarrow_{\text{im}}$ -normal form. By Confluence (Proposition 3.4) we conclude that $\mathbf{S}_*(t) = \mathbf{S}_*(s)$.

Second, suppose that $t \triangleright s$. By the fact that full simplification yields the normal form of a term (Proposition 3.12), we know that $t \rightarrow_{\text{im}}^* \mathbf{S}_*(t)$, where $\mathbf{S}_*(t)$ is in $\rightarrow_{\text{im}}$ -normal form. Since forgetful reduction commutes with reduction (Lemma 3.17), we have that there exists a term $u \in \Lambda_{\cap}^{\text{im}}$ such that $s \rightarrow_{\text{im}}^* u$ and $\mathbf{S}_*(t) \triangleright^+ u$. By Remark 3.14, since $\mathbf{S}_*(t)$ is in $\rightarrow_{\text{im}}$ -normal form, u is also in $\rightarrow_{\text{im}}$ -normal form. Moreover, $s \rightarrow_{\text{im}}^* \mathbf{S}_*(s)$ where $\mathbf{S}_*(s)$ is in $\rightarrow_{\text{im}}$ -normal form by Proposition 3.12. So by Confluence (Proposition 3.4) we have that $\mathbf{S}_*(s) = u$. Then we have that $\mathbf{S}_*(t) \triangleright^+ u = \mathbf{S}_*(s)$, as required. $\square$

Definition 3.19 (The $\mathcal{W}$ -measure). The measure of a term $t \in \Lambda_{\cap}^i$ is defined as the weight of its full simplification, where we recall that the weight is the number of wrappers in a term:

$$\mathcal{W}(t) := w(\mathbf{S}_*(t))$$

Theorem 3.20 (The $\mathcal{W}$ -measure is decreasing). *Let $t, s \in \Lambda_{\cap}^{\downarrow}$ be typable terms of the $\Lambda_{\cap}^{\downarrow}$ -calculus (without wrappers). If $t \rightarrow_{\downarrow} s$ then $\mathcal{W}(t) > \mathcal{W}(s)$.*

Proof. Let $t \rightarrow_{\downarrow} s$ and consider the corresponding step $t \rightarrow_{\text{im}} s'$. By the reduce/forget lemma (3.16) we have that $s' \triangleright s$. By Lemma 3.18 (1) we have that $\mathbf{S}_*(t) = \mathbf{S}_*(s')$. By Lemma 3.18 (2) we have that $\mathbf{S}_*(s') \triangleright^+ \mathbf{S}_*(s)$. Hence $\mathbf{S}_*(t) \triangleright^+ \mathbf{S}_*(s)$. By Remark 3.15 we conclude that $\mathcal{W}(t) = \mathbf{w}(\mathbf{S}_*(t)) > \mathbf{w}(\mathbf{S}_*(s)) = \mathcal{W}(s)$. $\square$

Corollary 3.21. $\Lambda_{\cap}^{\downarrow}$ is strongly normalizing.

4 Conclusion

In this section we provide a proof of the fact that $\Lambda_{\cap}^{\circ}$ characterizes strong normalization, based on the correspondence between it and $\Lambda_{\cap}^{\downarrow}$. This proof justifies the design of $\Lambda_{\cap}^{\circ}$, showing that it is a variant of the original system in [12], w.r.t. the typability power. Note that we make use of $\Lambda_{\cap}^{\downarrow}$ to prove the second item for uniformity purposes; it can also be proved entirely within $\Lambda_{\cap}^{\circ}$.

Corollary 4.1. $\Lambda_{\cap}^{\circ}$ characterizes strong normalization.

Proof. • Let $M \in \Lambda$ be a λ -term typable in the $\Lambda_{\cap}^{\circ}$ system, *i.e.* such that $\Gamma \vdash_{\circ} M : A$. By the Correspondence Theorem (2.24.1), there is a term $t \in \Lambda_{\cap}^{\downarrow}$ typable in the $\Lambda_{\cap}^{\downarrow}$ system such that $\Gamma \vdash_{\downarrow} t : A$ and $t \sqsubset M$. By Corollary 3.21, t is strongly normalizing. From the Simulation Theorem (2.26), we can observe that an infinite reduction sequence $M \rightarrow_{\beta} M_1 \rightarrow_{\beta} M_2 \rightarrow_{\beta} \dots$ would induce an infinite reduction sequence $t \rightarrow_{\downarrow}^+ t_1 \rightarrow_{\downarrow}^+ t_2 \rightarrow_{\downarrow}^+ \dots$, and this would contradict strong normalization of $\Lambda_{\cap}^{\downarrow}$. Hence M is strongly normalizing.

- Let $M \in \Lambda$ be strongly normalizing. By Lemma 2.28, there is $t \in \Lambda_{\cap}^{\downarrow}$ such that $t \sqsubset M$ and $\Gamma \vdash_{\downarrow} t : A$, for some Γ and A . Then t is uniform, and, by Theorem 2.24.2, $\Gamma \vdash_{\downarrow} t : A$ implies $\Gamma \vdash_{\circ} t^{\circ} : A$. Since $t^{\circ} = M$, the proof is given. $\square$

As we said in Section 1, this is not a completely new result. The novelty of our work comes from the technique we propose and the simplicity of its output. The strong normalization property for idempotent intersection type systems has already been proved in various papers using both semantical and syntactical approaches, and the fact that $\Lambda_{\cap}^{\circ}$ is a variant of them can be considered folklore. The semantical approach relies on arguments like computability or reducibility candidates (*e.g.* [29, 5]), while the syntactical ones are based on a measure that decreases with the β -reduction; as far as we know, there are three other syntactical proofs [22, 9, 11]. Here we supply a further syntactical proof, which uses as key ingredient a Church version of idempotent intersection types that has good proof-theoretical properties. The notion of measure obtained is simpler than [22, 9], being a natural number, and operates directly within the intersection system, allowing for refinement where measures based on the translation in [11] are constrained.

In the future, we would like to explore the possibility of providing an exact decreasing measure, *i.e.* a measure whose result is an exact upper bound for the length of the longest reduction chain starting from the term. While this refinement can be easily done for simple types, through a modification of the method in [2], the extension to intersection types presents challenges. In this case, the number of reduction steps in the derivation tree in general is bigger than the number of β -reductions in the term, so it would be necessary to change the operational behaviour of $\Lambda_{\cap}^{\downarrow}$.

Moreover, it would be interesting to study if the techniques given in this paper can be adapted to intersection type systems that characterize solvable terms, such as that of [14], for which no syntactic proofs of normalization have been proposed, as far as we know.

References

- [1] Franco Barbanera, Mariangiola Dezani-Ciancaglini, Ugo de'Liguoro, and Betti Venneri. YACC: yet another church calculus - A birthday present for herman inspired by his supervisor activity. In Venanzio Capretta, Robbert Krebbers, and Freek Wiedijk, editors, *Logics and Type Systems in Theory and Practice - Essays Dedicated to Herman Geuvers on The Occasion of His 60th Birthday*, volume 14560 of *Lecture Notes in Computer Science*, pages 17–35. Springer, 2024.
- [2] Pablo Barenbaum and Cristian Sottile. Two decreasing measures for simply typed λ -terms. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction, FSCD 2023, July 3-6, 2023, Rome, Italy*, volume 260 of *LIPICs*, pages 11:1–11:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. URL: <https://doi.org/10.4230/LIPICs.FSCD.2023.11>, doi:10.4230/LIPICs.FSCD.2023.11.
- [3] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, 1984.
- [4] Henk Barendregt, Mario Coppo, and Mariangiola Dezani-Ciancaglini. A filter lambda model and the completeness of type assignment. *J. Symb. Log.*, 48(4):931–940, 1983. doi:10.2307/2273659.
- [5] Henk P. Barendregt, Wil Dekkers, and Richard Statman. *Lambda Calculus with Types*. Lambda Calculus with Types. Cambridge University Press, 2013.
- [6] Henk P. Barendregt and Giulio Manzonetto. Turing’s contributions to lambda calculus. In B. Cooper and J. van Leeuwen, editors, *Alan Turing - His Work and Impact*, pages 139–143. Elsevier, 2013.
- [7] Alexis Bernadet and Stéphane Lengrand. Complexity of strongly normalising *lambda*-terms via non-idempotent intersection types. In *FOSSACS 2011*, pages 88–107, 2011.
- [8] Viviana Bono, Betti Venneri, and Lorenzo Bettini. A typed lambda calculus with intersection types. *Theoretical Computer Science*, 398(1-3):95–113, 2008. doi:10.1016/j.tcs.2008.01.046.
- [9] Gérard Boudol. On strong normalization in the intersection type discipline. In Martin Hofmann, editor, *Typed Lambda Calculi and Applications, 6th International Conference, TLCA 2003, Valencia, Spain, June 10-12, 2003, Proceedings*, volume 2701 of *Lecture Notes in Computer Science*, pages 60–74. Springer, 2003. doi:10.1007/3-540-44904-3_5.
- [10] Antonio Bucciarelli, Delia Kesner, and Daniel Ventura. Strong normalization through intersection types and memory. In Mario R. F. Benevides and René Thiemann, editors, *Proceedings of the Tenth Workshop on Logical and Semantic Frameworks, with Applications, LSFA 2015, Natal, Brazil, August 31 - September 1, 2015*, volume 323 of *Electronic Notes in Theoretical Computer Science*, pages 75–91. Elsevier, 2015. URL: <https://doi.org/10.1016/j.entcs.2016.06.006>, doi:10.1016/J.ENTCS.2016.06.006.
- [11] Antonio Bucciarelli, Adolfo Piperno, and Ivano Salvo. Intersection types and lambda-definability. *Math. Struct. Comput. Sci.*, 13(1):15–53, 2003. doi:10.1017/S0960129502003833.
- [12] Mario Coppo and Mariangiola Dezani-Ciancaglini. An extension of the basic functionality theory for the λ -calculus. *Notre Dame J. Formal Log.*, 21(4):685–693, 1980. URL: <https://doi.org/10.1305/ndjfl/1093883253>, doi:10.1305/NDJFL/1093883253.
- [13] Mario Coppo, Mariangiola Dezani-Ciancaglini, and Patrick Sallé. Functional characterization of some semantic equalities inside lambda-calculus. In Hermann A. Maurer, editor, *Automata, Languages and Programming, 6th Colloquium, Graz, Austria, July 16-20, 1979, Proceedings*, volume 71 of *Lecture Notes in Computer Science*, pages 133–146. Springer, 1979. doi:10.1007/3-540-09510-1_11.

- [14] Mario Coppo, Mariangiola Dezani-Ciancaglini, and Betti Venneri. Functional characters of solvable terms. *Math. Log. Q.*, 27(2-6):45–58, 1981. URL: <https://doi.org/10.1002/malq.19810270205>, doi:10.1002/MALQ.19810270205.
- [15] Daniel de Carvalho. Execution time of lambda-terms via denotational semantics and intersection types. *CoRR*, abs/0905.4251, 2009.
- [16] Philippe de Groote. The conservation theorem revisited. In Marc Bezem and Jan Friso Groote, editors, *Typed Lambda Calculi and Applications, International Conference on Typed Lambda Calculi and Applications, TLCA '93, Utrecht, The Netherlands, March 16-18, 1993, Proceedings*, volume 664 of *Lecture Notes in Computer Science*, pages 163–178. Springer, 1993. doi:10.1007/BFb0037105.
- [17] Roel de Vrijer. Exactly estimating functionals and strong normalization. In *Indagationes Mathematicae (Proceedings)*, volume 90, pages 479–493. North-Holland, 1987.
- [18] Robin O. Gandy. An early proof of normalization by A.M. Turing. In J.P. Seldin and J.R. Hindley, editors, *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 453–455. Academic Press, 1980.
- [19] Robin O. Gandy. Proofs of strong normalization. In J.P. Seldin and J.R. Hindley, editors, *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 457–477. Academic Press, 1980.
- [20] Philippa Gardner. Discovering needed reductions using type theory. In Masami Hagiya and John C. Mitchell, editors, *Theoretical Aspects of Computer Software, International Conference TACS '94, Sendai, Japan, April 19-22, 1994, Proceedings*, volume 789 of *Lecture Notes in Computer Science*, pages 555–574. Springer, 1994. doi:10.1007/3-540-57887-0_115.
- [21] A. J. Kfoury. A linearization of the lambda-calculus and consequences. *J. Log. Comput.*, 10(3):411–436, 2000. URL: <https://doi.org/10.1093/logcom/10.3.411>, doi:10.1093/LOGCOM/10.3.411.
- [22] A. J. Kfoury and J. B. Wells. New notions of reduction and non-semantic proofs of beta-strong normalization in typed lambda-calculi. In *Proceedings, 10th Annual IEEE Symposium on Logic in Computer Science, San Diego, California, USA, June 26-29, 1995*, pages 311–321. IEEE Computer Society, 1995. doi:10.1109/LICS.1995.523266.
- [23] Jan Willem Klop. *Combinatory Reduction Systems*. PhD thesis, Utrecht University, 1980.
- [24] Jean-Louis Krivine. *Lambda-calculus, types and models*. Ellis Horwood series in computers and their applications. Masson, 1993.
- [25] Robert Pieter Nederpelt Lazarom. *Strong normalization in a typed lambda calculus with lambda structured types*. PhD thesis, TU Eindhoven, 1973.
- [26] Luigi Liquori and Simona Ronchi Della Rocca. Intersection-types à la church. *Inf. Comput.*, 205(9):1371–1386, 2007. URL: <https://doi.org/10.1016/j.ic.2007.03.005>, doi:10.1016/J.IC.2007.03.005.
- [27] Peter Møller Neergaard and Morten Heine Sørensen. Conservation and uniform normalization in lambda calculi with erasing reductions. *Inf. Comput.*, 178(1):149–179, 2002. doi:10.1006/inco.2002.3153.
- [28] Luca Paolini, Mauro Piccolo, and Simona Ronchi Della Rocca. Essential and relational models. *Math. Struct. Comput. Sci.*, 27(5):626–650, 2017. doi:10.1017/S0960129515000316.
- [29] G. Pottinger. A type assignment for the strong normalizable lambda-terms. In J.R. Hindley and J.P. Seldin, editors, *To H.B. Curry: Essays on combinatory logic, lambda calculus and formalisms*, pages 561–577. Academic Press, 1980. doi:10.1109/LICS.1995.523266.

- [30] Simona Ronchi Della Rocca and Luca Paolini. *The Parametric Lambda Calculus - A Metamodel for Computation*. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004. doi:10.1007/978-3-662-10394-4.
- [31] Steffen van Bakel. Complete restrictions of the intersection type discipline. *Theor. Comput. Sci.*, 102:135–163, 1992.

A Technical appendix

A.1 Proofs of Section 2 — An intrinsically typed presentation of idempotent intersection types

In this section we give detailed proofs of the results about the $\Lambda_{\cap}^i$ -calculus stated in Section 2.

A.1.1 Subject reduction

Proposition 2.18 (Subject reduction). *If $t \rightarrow_i t'$ and $\Gamma \vdash_i t : A$ then $\Gamma \vdash_i t' : A$.*

Proof. We generalize the statement, proving also that if $\vec{t} \rightarrow_i \vec{t}'$ and $\Gamma \Vdash_i \vec{t} : \vec{A}$ then $\Gamma \Vdash_i \vec{t}' : \vec{A}$. The proof proceeds by induction on the definition of $\rightarrow_i$.

1. If t is a redex, i.e, $t = (\lambda x^{\vec{B}}. s) \vec{u}$, then the derivation proving $\Gamma \vdash_i t : A$ is:

$$\frac{\frac{\Gamma, x : \vec{B} \vdash_i s : A}{\Gamma \vdash_i \lambda x^{\vec{B}}. s : \vec{B} \rightarrow A} \quad \frac{}{\Gamma \Vdash_i \vec{u} : \vec{B}}}{\Gamma \vdash_i (\lambda x^{\vec{B}}. s) \vec{u} : A}$$

By Substitution Lemma 2.16, $\Gamma, x : \vec{B} \vdash_i s : A$ implies $\Gamma \vdash_i s[x^{\vec{B}} := \vec{u}] : A$.

Otherwise, it can be $t = \lambda x^A. s$, where $s \rightarrow_i s'$, or $t = s \vec{u}$, where either $s \rightarrow_i s'$ or $\vec{u} \rightarrow_i \vec{u}'$; in all these cases the proof follows by IH.

2. $\vec{t} \rightarrow_i \vec{t}'$ means $\vec{t} = \{t_1, \dots, t_n\}$, and $\vec{t}' = \{t'_1, \dots, t'_n\}$, and there is i such that $t_i \rightarrow_i t'_i$, and $t'_j = t_j$, for $j \neq i$. Then then proof follows by IH. $\square$

A.1.2 Confluence

Confluence will be proved by the methodology of parallel reduction.

Definition A.1. 1. The parallel reduction $\Rightarrow_i$ is defined on terms and set-terms by the following rules, closed under contexts.

$$\begin{array}{llll} x^A & \Rightarrow_i & x^A & \\ \lambda x^{\vec{A}}. t & \Rightarrow_i & \lambda x^{\vec{A}}. t' & \text{if } t \Rightarrow_i t' \\ t \vec{s} & \Rightarrow_i & t' \vec{s}' & \text{if } t \Rightarrow_i t' \text{ and } \vec{s} \Rightarrow_i \vec{s}' \\ (\lambda x^{\vec{A}}. t) \vec{s} & \Rightarrow_i & t'[x^{\vec{A}} := \vec{s}'] & \text{if } t \Rightarrow_i t' \text{ and } \vec{s} \Rightarrow_i \vec{s}' \\ \{t_1, \dots, t_n\} & \Rightarrow_i & \{t'_1, \dots, t'_n\} & \text{if } t_i \Rightarrow_i t'_i (1 \leq i \leq n) \end{array}$$

Note that $\Rightarrow_i$ is non-deterministic, since, when the term is a $\rightarrow_i$ redex, either the third or the forth rule can be applied. Roughly speaking, $\Rightarrow_i$ corresponds to reduce simultaneously a subset of the visible redexes by the $\rightarrow_i$ reduction.

2. The complete development of a term t , is the following function, which denotes the result of the simultaneous reduction of all visible redexes in a term.

$$\begin{array}{ll} \mathcal{C}(x^A) & := x^A \\ \mathcal{C}(\lambda x^{\vec{A}}. t) & := \lambda x^{\vec{A}}. \mathcal{C}(t) \\ \mathcal{C}((\lambda x^{\vec{A}}. t) \vec{s}) & := \mathcal{C}(t)[x^{\vec{A}} := \mathcal{C}(\vec{s})] \\ \mathcal{C}(t \vec{s}) & := \mathcal{C}(t) \mathcal{C}(\vec{s}) \text{ if } t \text{ is not an abstraction} \\ \mathcal{C}(\{t_1, \dots, t_n\}) & := \{\mathcal{C}(t_1), \dots, \mathcal{C}(t_n)\} \end{array}$$

The $\Rightarrow_i$ reduction enjoys the following properties, whose proof is easy.

Proposition A.2. 1. $t \rightarrow_i t'$ implies $t \Rightarrow_i t'$.

2. $t \Rightarrow_i t'$ implies $t \rightarrow_i^* t'$.

3. $\rightarrow_i^*$ is the transitive closure of $\Rightarrow_i$.

Lemma A.3 (Substitution). *Let $t \Rightarrow_i t'$ and $\vec{s} \Rightarrow_i \vec{s}'$. Then $t[x^A := \vec{s}]$ implies $t'[x^A := \vec{s}']$.*

Proof. By induction on t . □

Lemma A.4. $t \Rightarrow_i t'$ implies $t' \Rightarrow_i \mathcal{C}(t)$.

Proof. By induction on t . Let $t = (\lambda x^A. u) \vec{s}$. So t' is either $(\lambda x^A. u') \vec{s}'$ or $u'[x^A := \vec{s}']$, where $\vec{s} \Rightarrow_i \vec{s}'$ and $u \Rightarrow_i u'$. By IH, $u' \Rightarrow_i \mathcal{C}(u)$ and $\vec{s}' \Rightarrow_i \mathcal{C}(\vec{s})$. In both cases, $t' \rightarrow_i \mathcal{C}(u)[x^A := \mathcal{C}(\vec{s})] = \mathcal{C}(t)$, in the former case by IH, in the latter by IH and Lemma A.3. The other cases come directly from IH. □

The $\Rightarrow_i$ reduction enjoys the diamond property, i.e.,

Lemma A.5. *If $t \Rightarrow_i t_1$ and $t \Rightarrow_i t_2$, then there is t_3 such that $t_1 \Rightarrow_i t_3$ and $t_2 \Rightarrow_i t_3$.*

Proof. By Lemma A.4, $t_3 = \mathcal{C}(t)$. □

Proposition 2.19 (Confluence). *If $t \rightarrow_i^* s$ and $t \rightarrow_i^* u$ then there is r such that $s \rightarrow_i^* r$ and $u \rightarrow_i^* r$.*

Proof. By Proposition A.2.3 and Lemma A.5. □

A.1.3 Strong Normalization typability

To prove this we need first to prove that a partial subject expansion holds, when the redex to be expanded is in head position.

Lemma A.6 (Head subject expansion). *If $\Gamma \vdash_i t[x^{\vec{A}} := \vec{s}] \vec{s}_1 \dots \vec{s}_n : B$ and $\Gamma \Vdash_i \vec{s} : \vec{A}$, then $\Gamma \vdash_i (\lambda x^A. t) \vec{s} \vec{s}_1 \dots \vec{s}_n : B$.*

Proof. By induction on n . Let $n = 0$, so $\Gamma \vdash_i t[x^{\vec{A}} := \vec{s}] : B$. We continue by induction on t .

The interesting case is that of $t = y^B$, where $B \in \vec{A}$. Then $\Gamma, x^{\vec{A}} \vdash_i x^B : B$, by rule (i-var), and $\Gamma \vdash_i (\lambda x^{\vec{A}}. x^B) \vec{s} : B$ by rules (i-I $\rightarrow$) and (i-E $\rightarrow$). The remaining cases are straightforward, by IH and substitution definition.

Let $n > 0$. Let $\Gamma \vdash_i t[x^{\vec{A}} := \vec{s}] \vec{s}_1 \dots \vec{s}_n \vec{u} : B$ and $\Gamma \Vdash_i \vec{s} : \vec{A}$. Then, by the rules of the system, $\Gamma \vdash_i t[x^{\vec{A}} := \vec{s}] \vec{s}_1 \dots \vec{s}_n \vec{u} : B$ comes from $\Gamma \vdash_i t[x^{\vec{A}} := \vec{s}] \vec{s}_1 \dots \vec{s}_n : \vec{C} \rightarrow B$ and $\Gamma \vdash_i \vec{u} : \vec{C}$, for some $\vec{C}$. By IH, $\Gamma \vdash_i (\lambda x^A. t) \vec{s} \vec{s}_1 \dots \vec{s}_n : \vec{C} \rightarrow B$, and the result follows by rule (i-E $\rightarrow$). □

Lemma 2.28 (Strong Normalization typability). *Let $M \in \Lambda$ be strongly normalizing. Then there is $t \in \Lambda_\cap^i$ such that $t \sqsubset M$ and $\Gamma \vdash_i t : A$, for some Γ and A .*

Proof. The proof relies on the following inductive definition of the strongly normalizing terms (SN).

$$\begin{array}{c} \frac{M_i \in SN \quad 1 \leq i \leq n}{x M_1 \dots M_n \in SN} \text{ SN1} \qquad \frac{M \in SN}{\lambda x. M \in SN} \text{ SN2} \\[1.5em] \frac{M[x := N] M_1 \dots M_n \in SN \quad N \in SN}{(\lambda x. M) N M_1 \dots M_n \in SN} \text{ SN3} \end{array}$$

Let us consider rule (SN1). By induction there are t_i such that $t_i \sqsubset M_i$ and $\Gamma_i \vdash_i t_i : A_i$, so $\Gamma_i \Vdash_i \vec{t}_i : \vec{A}_i$. So, by Lemma 2.10, and rule (i-E $\rightarrow$), $(\bigcup_{1 \leq i \leq n} \Gamma_i) \cup x : \{\vec{A}_1 \rightarrow \dots \rightarrow \vec{A}_n \rightarrow B\} \vdash_i x^{\{\vec{A}_1 \rightarrow \dots \rightarrow \vec{A}_n \rightarrow B\}} \vec{t}_1 \dots \vec{t}_n : B$. Let us consider rule (SN3). Let $M[x := N] M_1 \dots M_n$ and $N \in SN$. By IH, $\exists t, s, \Gamma, \Gamma', A, B$ such that $t \sqsubset M[x := N] M_1 \dots M_n$, $s \sqsubset N$ and $\Gamma \vdash_i t : B$, $\Gamma' \vdash_i s : A$. Note that $\Gamma' \vdash_i s : A$ implies $\Gamma' \Vdash_i \vec{s} : \vec{A}$, where $\vec{s} = \{s\}$ and $\vec{A} = \{A\}$. $\Gamma \vdash_i t : B$ and $t \sqsubset M[x := N] M_1 \dots M_n$ imply $t = t' t_1 \dots t_n$, where $t_i \sqsubset M_i$ and t' can be written as $t''[x^{\vec{A}} := \vec{s}]$. By Lemma 2.10, $\Gamma \cup \Gamma' \vdash_i t''[x^{\vec{A}} := \vec{s}] t_1 \dots t_n : B$ and $\Gamma \cup \Gamma' \Vdash_i \vec{s} : \vec{A}$, and the result follows by Lemma A.6. The case of rule (SN2) follows directly by IH. □

Note that we make use of $\Lambda_\cap^i$ in this proof for uniformity purposes. It can also be proved entirely within $\Lambda_\cap^\circ$.

A.2 Proofs of Section 3 — Strong normalization via a decreasing measure

In this section we give detailed proofs of the results about the $\Lambda_{\cap}^{\text{im}}$ -calculus stated in Section 3.

A.2.1 Subject reduction

Lemma A.7 (Substitution).

1. Let $\Gamma, x : \vec{A} \vdash_{\text{im}} t : B$ and $\Gamma \Vdash_{\text{im}} \vec{s} : \vec{A}$. Then $\Gamma \vdash_{\text{im}} t[x := \vec{s}] : B$.
2. Let $\Gamma, x : \vec{A} \Vdash_{\text{im}} \vec{t} : \vec{B}$ and $\Gamma \Vdash_{\text{im}} \vec{s} : \vec{A}$. Then $\Gamma \Vdash_{\text{im}} \vec{t}[x := \vec{s}] : \vec{B}$.

Proof. We prove both items simultaneously by a mutual induction on t and $\vec{t}$.

1. $t = y^B$: then $\Gamma, x : \vec{A} \vdash_{\text{im}} y : B$ concludes with **im-var**. If $y = x$, then $y[x := \vec{s}] = \vec{s}$, which by hypothesis has type B under Γ . If $y \neq x$, then $y[x := \vec{s}] = y$, which by hypothesis and strengthening has type B under Γ .
2. $t = \lambda y^{\vec{C}}. t'$: then $\Gamma, x : \vec{A} \vdash_{\text{im}} \lambda y^{\vec{C}}. t' : \vec{C} \rightarrow D$ concludes with **im-I $\rightarrow$** , with $B = \vec{C} \rightarrow D$. By inversion, we have $\Gamma, x : \vec{A}, y : \vec{C} \vdash_{\text{im}} t' : D$. By IH, $\Gamma, y : \vec{C} \vdash_{\text{im}} t'[x := \vec{s}] : D$, hence by **im-I $\rightarrow$** it follows that $\Gamma \vdash_{\text{im}} \lambda y^{\vec{C}}. t'[x := \vec{s}] : \vec{C} \rightarrow D$. By definition of substitution, $(\lambda y^{\vec{C}}. t')[x := \vec{s}] = \lambda y^{\vec{C}}. t'[x := \vec{s}]$, hence we conclude that $\Gamma \vdash_{\text{im}} (\lambda y^{\vec{C}}. t')[x := \vec{s}] : \vec{C} \rightarrow D$.
3. $t = t_1 t_2$: then $\Gamma, x : \vec{A} \vdash_{\text{im}} t_1 t_2 : B$ concludes with **im-E $\rightarrow$** , so by inversion we have that $\Gamma, x : \vec{A} \vdash_{\text{im}} t_1 : \vec{C} \rightarrow B$ and $\Gamma, x : \vec{A} \vdash_{\text{im}} t_2 : \vec{C}$. By IH, $\Gamma \vdash_{\text{im}} t_1[x := \vec{s}] : \vec{C} \rightarrow B$ and $\Gamma \Vdash_{\text{im}} t_2[x := \vec{s}] : \vec{C}$. By **im-E $\rightarrow$** it follows that $\Gamma \vdash_{\text{im}} t_1[x := \vec{s}] t_2[x := \vec{s}] : B$. By definition of substitution $(t_1 t_2)[x := \vec{s}] = t_1[x := \vec{s}] t_2[x := \vec{s}]$, hence we conclude that $\Gamma \vdash_{\text{im}} (t_1 t_2)[x := \vec{s}] : B$.
4. $t = t_1 \langle \vec{t}_2 \rangle$: then $\Gamma, x : \vec{A} \vdash_{\text{im}} t_1 \langle \vec{t}_2 \rangle : B$ concludes with **im-wrap**, so by inversion we have that $\Gamma, x : \vec{A} \vdash_{\text{im}} t_1 : B$ and $\Gamma, x : \vec{A} \Vdash_{\text{im}} \vec{t}_2 : \vec{C}$. By IH, $\Gamma \vdash_{\text{im}} t_1[x := \vec{s}] : B$ and $\Gamma \Vdash_{\text{im}} \vec{t}_2[x := \vec{s}] : \vec{C}$. By **im-wrap** it follows that $\Gamma \vdash_{\text{im}} t_1[x := \vec{s}] \langle \vec{t}_2[x := \vec{s}] \rangle : B$. By definition of substitution $(t_1 \langle \vec{t}_2 \rangle)[x := \vec{s}] = t_1[x := \vec{s}] \langle \vec{t}_2[x := \vec{s}] \rangle$, hence we conclude that $\Gamma \vdash_{\text{im}} (t_1 \langle \vec{t}_2 \rangle)[x := \vec{s}] : B$.
5. $t = \{t_1, \dots, t_n\}$: then $\Gamma, x : \vec{A} \Vdash_{\text{im}} \{t_1, \dots, t_n\} : \vec{B}$ concludes with **im-many**, so by inversion we have that $\Gamma, x : \vec{A} \vdash_{\text{im}} t_i : B_i$ for each $i \in 1..n$, where $B_i \in \vec{B}$. By IH, $\Gamma \vdash_{\text{im}} t_i[x := \vec{s}] : B_i$ for each $i \in 1..n$. By **im-many** it follows that $\Gamma \Vdash_{\text{im}} \{t_1[x := \vec{s}], \dots, t_n[x := \vec{s}]\} : \vec{B}$. By definition of substitution, $\{t_1, \dots, t_n\}[x := \vec{s}] = \{t_1[x := \vec{s}], \dots, t_n[x := \vec{s}]\}$, hence we conclude that $\Gamma \Vdash_{\text{im}} \{t_1, \dots, t_n\}[x := \vec{s}] : \vec{B}$. $\square$

Proposition 3.3 (Subject reduction). *If $t \rightarrow_{\text{im}} t'$ and $\Gamma \vdash_{\text{im}} t : A$, then $\Gamma \vdash_{\text{im}} t' : A$.*

Proof. Simultaneously by induction on the reduction relation $\rightarrow_{\text{im}}$.

1. In the case $t = (\lambda x^{\vec{B}}. s) \text{L} \vec{u} \rightarrow_{\text{im}} s[x := \vec{u}] \langle \vec{u} \rangle \text{L} = t'$ we have that t must be typed with the following derivation scheme, where n is the amount of wrappers in L :

$$\begin{array}{c}
 \frac{\Gamma, x : \vec{B} \vdash_{\text{im}} s : C}{\Gamma \vdash_{\text{im}} \lambda x^{\vec{B}}. s : \vec{B} \rightarrow C} \text{im-I} \rightarrow \quad \Pi_1 \quad \text{im-wrap} \\
 \vdots \\
 \frac{\Gamma \vdash_{\text{im}} \lambda x^{\vec{B}}. s \text{L} : \vec{B} \rightarrow C \quad \Gamma \vdash_{\text{im}} \vec{u} : \vec{B}}{\Gamma \vdash_{\text{im}} (\lambda x^{\vec{B}}. s) \text{L} \vec{u} : C} \Pi_n \quad \text{im-wrap} \quad \text{im-E} \rightarrow
 \end{array}$$

Hence from Lemma A.7, since $\Gamma, x : \vec{B} \vdash_{\text{im}} s : C$ and $\Gamma \vdash_{\text{im}} \vec{u} : \vec{B}$, we obtain $\Gamma \vdash_{\text{im}} s[x := \vec{u}] : C$. Then from the corresponding $n + 1$ applications of the **im-wrap** we obtain $\Gamma \vdash_{\text{im}} s[x := \vec{u}] \langle \vec{u} \rangle \text{L} : C$.

2. If $t = \lambda x^{\vec{B}}. s \rightarrow_{\text{im}} \lambda x^{\vec{B}}. s' = t'$ with $s \rightarrow_{\text{im}} s'$, then $A = \vec{B} \rightarrow C$ and $\Gamma \vdash_{\text{im}} \lambda x^{\vec{B}}. s : \vec{B} \rightarrow C$. By inversion of $\text{im-I} \rightarrow$ it follows that $\Gamma, x : \vec{B} \vdash_{\text{im}} s : C$. Then by IH we have that $\Gamma, x : \vec{B} \vdash_{\text{im}} s' : C$, and by $\text{im-I} \rightarrow$ we conclude $\Gamma \vdash_{\text{im}} \lambda x^{\vec{B}}. s' : \vec{B} \rightarrow C$.
3. If $t = s \vec{u} \rightarrow_{\text{im}} s' \vec{u} = t'$ with $s \rightarrow_{\text{im}} s'$, then we have that $\Gamma \vdash_{\text{im}} s \vec{u} : A$ and, by inversion of $\text{im-E} \rightarrow$, it follows that $\Gamma \vdash_{\text{im}} s : \vec{B} \rightarrow A$ and $\Gamma \Vdash_{\text{im}} \vec{u} : \vec{B}$. Then by IH we have that $\Gamma \vdash_{\text{im}} s' : \vec{B} \rightarrow A$, and by $\text{im-E} \rightarrow$ we conclude $\Gamma \vdash_{\text{im}} s' \vec{u} : A$.
4. If $t = s \vec{u} \rightarrow_{\text{im}} s \vec{u}' = t'$ with $\vec{u} \rightarrow_{\text{im}} \vec{u}'$, then we have that $\Gamma \vdash_{\text{im}} s \vec{u} : A$ and, by inversion of $\text{im-E} \rightarrow$, it follows that $\Gamma \vdash_{\text{im}} s : \vec{B} \rightarrow A$ and $\Gamma \Vdash_{\text{im}} \vec{u} : \vec{B}$. Then by IH we have that $\Gamma \Vdash_{\text{im}} \vec{u}' : \vec{B}$, and by $\text{im-E} \rightarrow$ we conclude $\Gamma \vdash_{\text{im}} s \vec{u}' : A$.
5. If $\vec{t} = \{t_1, \dots, t_i, \dots, t_n\} \rightarrow_{\text{im}} \{t_1, \dots, t'_i, \dots, t_n\} = \vec{t}'$ with $i \in 1..n$, $n \geq 1$ and $t_i \rightarrow_{\text{im}} t'_i$, then we have that $\Gamma \Vdash_{\text{im}} \{t_1, \dots, t_i, \dots, t_n\} : \{A_1, \dots, A_i, \dots, A_n\}$. By inversion of rule im-many , it follows that $\Gamma \vdash_{\text{im}} t_1 : A_1, \dots, \Gamma \vdash_{\text{im}} t_i : A_i, \dots, \Gamma \vdash_{\text{im}} t_n : A_n$. Then by IH we have that $\Gamma \vdash_{\text{im}} t'_i : A_i$, and by im-many we conclude $\Gamma \Vdash_{\text{im}} \{t_1, \dots, t'_i, \dots, t_n\} : \{A_1, \dots, A_i, \dots, A_n\}$.
6. If $t = s \langle \vec{u} \rangle \rightarrow_{\text{im}} s' \langle \vec{u} \rangle = t'$ with $s \rightarrow_{\text{im}} s'$, then we have that $\Gamma \vdash_{\text{im}} s \langle \vec{u} \rangle : A$ and, by inversion of im-wrap , it follows that $\Gamma \vdash_{\text{im}} s : A$ and $\Gamma \Vdash_{\text{im}} \vec{u} : \vec{B}$. Then by IH we have that $\Gamma \vdash_{\text{im}} s' : A$, and by im-wrap we conclude $\Gamma \vdash_{\text{im}} s' \langle \vec{u} \rangle : A$.
7. If $t = s \langle \vec{u} \rangle \rightarrow_{\text{im}} s \langle \vec{u}' \rangle = t'$ with $\vec{u} \rightarrow_{\text{im}} \vec{u}'$, then we have that $\Gamma \vdash_{\text{im}} s \langle \vec{u} \rangle : A$ and, by inversion of im-wrap , it follows that $\Gamma \vdash_{\text{im}} s : A$ and $\Gamma \Vdash_{\text{im}} \vec{u} : \vec{B}$. Then by IH we have that $\Gamma \Vdash_{\text{im}} \vec{u}' : \vec{B}$, and by im-wrap we conclude $\Gamma \vdash_{\text{im}} s \langle \vec{u}' \rangle : A$. $\square$

A.2.2 Confluence

We extend the proof of Proposition 2.19 by adding the cases involving wrappers and modifying those involving i-redexes by im-redexes (*i.e.* allowing a list of wrappers L between the w -abstraction and the argument).

Definition A.8.

1. The parallel reduction $\Rightarrow_{\text{im}}$ is extended by modifying the β case to allow lists of wrappers, and by adding the congruence cases of wrappers and lists of wrappers:

$$\begin{array}{lll}
(\lambda x^{\vec{A}}. t) L \vec{s} & \Rightarrow_{\text{im}} & t'[x^{\vec{A}} := \vec{s}'] \langle \vec{s}' \rangle L' \quad \text{if } t \Rightarrow_{\text{im}} t', s \Rightarrow_{\text{im}} \vec{s}' \text{ and } L \Rightarrow_{\text{im}} L' \\
t \langle \vec{s} \rangle & \Rightarrow_{\text{im}} & t' \langle \vec{s}' \rangle \quad \text{if } t \Rightarrow_{\text{im}} t' \text{ and } \vec{s} \Rightarrow_{\text{im}} \vec{s}' \\
\Box \langle \vec{t}_1 \rangle \dots \langle \vec{t}_n \rangle & \Rightarrow_{\text{im}} & \Box \langle \vec{t}'_1 \rangle \dots \langle \vec{t}'_n \rangle \quad \text{if } \vec{t}_i \Rightarrow_{\text{im}} \vec{t}'_i \text{ for all } i \in 1..n
\end{array}$$

2. The complete development of a term t is extended by modifying the β case to allow lists of wrappers, and by adding the congruence cases of wrappers and lists of wrappers

$$\begin{aligned}
\mathbf{C}((\lambda x^{\vec{A}}. t) L \vec{s}) &:= \mathbf{C}(t)[x^{\vec{A}} := \mathbf{C}(\vec{s})] \langle \mathbf{C}(\vec{s}) \rangle \mathbf{C}(L) \\
\mathbf{C}(t \langle \vec{s} \rangle) &:= \mathbf{C}(t) \langle \mathbf{C}(\vec{s}) \rangle \\
\mathbf{C}(\Box \langle \vec{t}_1 \rangle \dots \langle \vec{t}_n \rangle) &:= \Box \langle \mathbf{C}(t_1) \rangle, \dots, \langle \mathbf{C}(t_n) \rangle
\end{aligned}$$

As well as $\Rightarrow_i$ reduction, $\Rightarrow_{\text{im}}$ reduction enjoys the following properties:

Proposition A.9 (Properties of $\Rightarrow_{\text{im}}$).

1. $t \rightarrow_{\text{im}} t'$ implies $t \Rightarrow_{\text{im}} t'$.
2. $t \Rightarrow_{\text{im}} t'$ implies $t \rightarrow_{\text{im}}^* t'$.
3. $\rightarrow_{\text{im}}^*$ is the transitive closure of $\Rightarrow_{\text{im}}$.

Lemma A.10 (Substitution). $t[y^A := \vec{s}][x^B := \vec{u}] = t[x^B := \vec{u}][y^A := \vec{s}[x^B := \vec{u}]]$ if $y \notin \text{fv}(\vec{u})$

Proof. By induction on t . □

Lemma A.11 (Compatibility of substitution and parallel reduction). *Let $t \Rightarrow_{\text{im}} t'$ and $\vec{s} \Rightarrow_{\text{im}} \vec{s}'$. Then $t[x^A := \vec{s}] \Rightarrow_{\text{im}} t'[x^A := \vec{s}']$.*

Proof. By induction on t , where the interesting case is that of $t = (\lambda y^A. u) \text{L} \vec{r}$. Then t' depends on whether the head im -redex is contracted or not. In both cases it holds that $u \Rightarrow_{\text{im}} u'$, $\vec{r} \Rightarrow_{\text{im}} \vec{r}'$, and $\text{L} \Rightarrow_{\text{im}} \text{L}'$.

1. If $t' = (\lambda y^A. u') \text{L}' \vec{r}'$, then, by definition of substitution and IH, we have that

$$\begin{aligned} ((\lambda y^A. u) \text{L} \vec{r})[x^A := \vec{s}] &= (\lambda y^A. u[x^A := \vec{s}]) \text{L}[x^A := \vec{s}] \vec{r}[x^A := \vec{s}] \\ &\Rightarrow_{\text{im}} (\lambda y^A. u'[x^A := \vec{s}']) \text{L}'[x^A := \vec{s}'] \vec{r}'[x^A := \vec{s}'] \\ &= ((\lambda y^A. u') \text{L}' \vec{r}') [x^A := \vec{s}'] \end{aligned}$$

2. If $t' = u'[y^A := \vec{r}'] \langle \vec{r}' \rangle \text{L}'$, then, by definition of substitution, IH, and Lemma A.10, we have that

$$\begin{aligned} ((\lambda y^A. u) \text{L} \vec{r})[x^A := \vec{s}] &= (\lambda y^A. u[x^A := \vec{s}]) \text{L}[x^A := \vec{s}] \vec{r}[x^A := \vec{s}] \\ &\Rightarrow_{\text{im}} u'[x^A := \vec{s}'] [y^A := \vec{r}'[x^A := \vec{s}']] \langle \vec{r}'[x^A := \vec{s}'] \rangle \text{L}'[x^A := \vec{s}'] \\ &= u'[y^A := \vec{r}'] [x^A := \vec{s}'] \langle \vec{r}'[x^A := \vec{s}'] \rangle \text{L}'[x^A := \vec{s}'] \\ &= (u'[y^A := \vec{r}'] \langle \vec{r}' \rangle \text{L}') [x^A := \vec{s}'] \end{aligned}$$

□

Lemma A.12 (Composition lemma for wrappers). *If $t \Rightarrow_{\text{im}} t'$ and $\text{L} \Rightarrow_{\text{im}} \text{L}'$, then $t\text{L} \Rightarrow_{\text{im}} t'\text{L}'$.*

Proof. Let $\text{L} = \square \langle \vec{t}_1 \rangle \dots \langle \vec{t}_n \rangle$. We proceed by induction on n .

1. If $n = 0$, then $t\text{L}\square = t \Rightarrow_{\text{im}} t' = t'\square = t'\text{L}'$.
2. If $n = k + 1$, then by IH we have that $t\square \langle \vec{t}_1 \rangle \dots \langle \vec{t}_k \rangle \Rightarrow_{\text{im}} t'\square \langle \vec{t}_1' \rangle \dots \langle \vec{t}_k' \rangle$, hence

$$\begin{aligned} t\text{L} &= t \langle \vec{t}_1 \rangle \dots \langle \vec{t}_k \rangle \langle \vec{t}_{k+1} \rangle \\ &= t \langle \vec{t}_1 \rangle \dots \langle \vec{t}_k \rangle \langle \vec{t}_{k+1} \rangle \\ &\Rightarrow_{\text{im}} t' \langle \vec{t}_1' \rangle \dots \langle \vec{t}_k' \rangle \langle \vec{t}_{k+1}' \rangle \\ &= t' \square \langle \vec{t}_1' \rangle \dots \langle \vec{t}_k' \rangle \langle \vec{t}_{k+1}' \rangle \\ &= t'\text{L}' \end{aligned}$$

□

Lemma A.13. $t \Rightarrow_i t'$ implies $t' \Rightarrow_i \text{C}(t)$.

Proof. By induction on t . The interesting case is that of $t = (\lambda x^A. u) \text{L} \vec{s}$. Then t' depends on whether the head im -redex is contracted or not. In both cases it holds that $u \Rightarrow_{\text{im}} u'$, $\vec{s} \Rightarrow_{\text{im}} \vec{s}'$, and $\text{L} \Rightarrow_{\text{im}} \text{L}'$.

1. If $t' = (\lambda x^A. u') \text{L}' \vec{s}'$, then by IH we have that

$$\begin{aligned} (\lambda x^A. u) \text{L} \vec{s} &\Rightarrow_{\text{im}} (\lambda x^A. u') \text{L}' \vec{s}' \\ &\Rightarrow_{\text{im}} (\lambda x^A. \text{C}(u)) \text{C}(\text{L}) \text{C}(\vec{s}) \\ &\rightarrow_{\text{im}} \text{C}(u)[x^A := \text{C}(\vec{s})] \langle \text{C}(\vec{s}) \rangle \text{C}(\text{L}) \\ &= \text{C}((\lambda x^A. u) \text{L} \vec{s}) \end{aligned}$$

2. If $t' = u'[x^A := \vec{s}'] \langle \vec{s}' \rangle \text{L}'$, then by IH and Lemma A.11 we have that

$$\begin{aligned} (\lambda x^A. u) \text{L} \vec{s} &\Rightarrow_{\text{im}} u'[x^A := \vec{s}'] \langle \vec{s}' \rangle \text{L}' \\ &\Rightarrow_{\text{im}} \text{C}(u)[x^A := \text{C}(\vec{s})] \langle \text{C}(\vec{s}) \rangle \text{C}(\text{L}) \\ &= \text{C}((\lambda x^A. u) \text{L} \vec{s}) \end{aligned}$$

The remaining cases come directly from IH. □

The $\Rightarrow_i$ reduction enjoys the diamond property, i.e.,

Lemma A.14. *If $t \Rightarrow_i t_1$ and $t \Rightarrow_i t_2$, then there is t_3 such that $t_1 \Rightarrow_i t_3$ and $t_2 \Rightarrow_i t_3$.*

Proof. By Lemma A.13, $t_3 = \mathcal{C}(t)$. □

Proposition 3.4 (Confluence). *If $t_1 \rightarrow_{\text{im}}^* t_2$ and $t_1 \rightarrow_{\text{im}}^* t_3$, there exists a term t_4 such that $t_2 \rightarrow_{\text{im}}^* t_4$ and $t_3 \rightarrow_{\text{im}}^* t_4$.*

Proof. By Lemma A.9.3 and Lemma A.14. □

A.2.3 Forgetful reduction

Lemma A.15 (Forgetful reduction commutes with reduction). *If $t_1 \rightarrow_{\text{im}}^* t_2$ and $t_1 \triangleright^+ t_3$ there exists t_4 such that $t_2 \triangleright^+ t_4$ and $t_3 \rightarrow_{\text{im}}^* t_4$. Graphically:*

$$\begin{array}{ccc} t_1 & \rightarrow_{\text{im}}^* & t_2 \\ \nabla^+ & & \nabla^+ \\ t_3 & \rightarrow_{\text{im}}^* & t_4 \end{array}$$

Proof. It suffices to prove a local commutation result, namely that if $t_1 \rightarrow_{\text{im}} t_2$ and $t_1 \triangleright t_3$ there exists t_4 such that $t_2 \triangleright^+ t_4$ and $t_3 \rightarrow_{\text{im}}^= t_4$. Graphically:

$$\begin{array}{ccc} t_1 & \rightarrow_{\text{im}} & t_2 \\ \nabla & & \nabla^+ \\ t_3 & \rightarrow_{\text{im}}^= & t_4 \end{array}$$

We proceed by induction on t_1 .

1. $t_1 = x^{\vec{A}}$: this case is impossible, since x does not reduce.
2. $t_1 = \lambda x^{\vec{A}}. t'_1$: then $t_2 = \lambda x^{\vec{A}}. t'_2$ and $t_3 = \lambda x^{\vec{A}}. t'_3$, with $t'_1 \rightarrow_{\text{im}} t'_2$ and $t'_1 \triangleright t'_3$. By IH there exists t'_4 s.t. $t'_3 \rightarrow_{\text{im}}^= t'_4$ and $t'_2 \triangleright^+ t'_4$. By congruence, the following holds

$$\begin{array}{ccc} \lambda x^{\vec{A}}. t'_1 & \rightarrow_{\text{im}} & \lambda x^{\vec{A}}. t'_2 \\ \nabla & & \nabla^+ \\ \lambda x^{\vec{A}}. t'_3 & \rightarrow_{\text{im}}^= & \lambda x^{\vec{A}}. t'_4 \end{array}$$

3. $t_1 = t_{11} \vec{t}_{12}$. We consider two subcases here, depending on t_{11} being a w-abstraction and on which redex is contracted.

- 3.1 $t_1 = (\lambda x^{\vec{A}}. t_{11}) \text{L} \vec{t}_{12}$ and the contracted redex is at head position. Then

$$\begin{array}{ccc} (\lambda x^{\vec{A}}. t_{11}) \text{L} \vec{t}_{12} & \rightarrow_{\text{im}} & t_{11} [x^{\vec{A}} := \vec{t}_{12}] \langle \vec{t}_{12} \rangle \text{L} \\ \nabla & & \\ (\lambda x^{\vec{A}}. t'_{11}) \text{L}' \vec{t}_{12} & & \end{array}$$

where since there is a single $\triangleright$ -step, only one of t'_{11} , L' , or $\vec{t}_{12}$, can be different from t_{11} , L , or $\vec{t}_{12}$, respectively. If the $\triangleright$ -step occurs in t_{11} or L , then the same step can be reproduced from t_2 .

Else, considering $\vec{t}_{12}$ can appear more than one time in t_2 , the $\triangleright$ -step may have to be reproduced several times, hence the $\triangleright^+$. Then we have that

$$\begin{array}{ccc} (\lambda x^{\vec{A}}. t_{11})L \vec{t}_{12} & \rightarrow_{\text{im}} & t_{11}[x^{\vec{A}} := \vec{t}_{12}]\langle \vec{t}_{12} \rangle L \\ \nabla & & \nabla^+ \\ (\lambda x^{\vec{A}}. t'_{11})L' \vec{t}_{12} & \rightarrow_{\text{im}} & t'_{11}[x^{\vec{A}} := \vec{t}_{12}]\langle \vec{t}_{12} \rangle L' \end{array}$$

3.2 t_{11} is not a w-abstraction or the contracted redex is not the one at head position. We consider two subcases.

3.2.1 Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc} t_{11}\vec{t}_{12} & \rightarrow_{\text{im}} & t'_{11}\vec{t}_{12} & t_{11}\vec{t}_{12} & \rightarrow_{\text{im}} & t_{11}\vec{t}'_{12} \\ \nabla & & \nabla^+ & \nabla & & \nabla^+ \\ t''_{11}\vec{t}_{12} & \rightarrow_{\text{im}}^= & t'''_{11}\vec{t}_{12} & t_{11}\vec{t}''_{12} & \rightarrow_{\text{im}}^= & t_{11}\vec{t}'''_{12} \end{array}$$

3.2.2 The steps occur in different subterms. Then the diagram can be closed immediately:

$$\begin{array}{ccc} t_{11}\vec{t}_{12} & \rightarrow_{\text{im}} & t_{11}\vec{t}'_{12} \\ \nabla & & \nabla \\ t'_{11}\vec{t}_{12} & \rightarrow_{\text{im}} & t'_{11}\vec{t}'_{12} \end{array}$$

$$\begin{array}{ccc} t_{11}\vec{t}_{12} & \rightarrow_{\text{im}} & t_{11}\vec{t}_{12} \\ \nabla & & \nabla \\ t_{11}\vec{t}'_{12} & \rightarrow_{\text{im}} & t_{11}\vec{t}'_{12} \end{array}$$

4. $t_1 = t_{11}\langle \vec{t}_{12} \rangle$. We consider three general subcases:

4.1 Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc} t_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t'_{11}\langle \vec{t}_{12} \rangle & t_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t_{11}\langle \vec{t}'_{12} \rangle \\ \nabla & & \nabla^+ & \nabla & & \nabla^+ \\ t''_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}}^= & t'''_{11}\langle \vec{t}_{12} \rangle & t_{11}\langle \vec{t}'_{12} \rangle & \rightarrow_{\text{im}}^= & t_{11}\langle \vec{t}'''_{12} \rangle \end{array}$$

$$\begin{array}{ccc} t_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t'_{11}\langle \vec{t}_{12} \rangle \\ \nabla & & \nabla \\ t_{11}\langle \vec{t}'_{12} \rangle & \rightarrow_{\text{im}} & t'_{11}\langle \vec{t}'_{12} \rangle \end{array}$$

4.2 The steps occur in different subterms. Then the diagram can be closed immediately:

$$\begin{array}{ccc} t_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t_{11}\langle \vec{t}'_{12} \rangle \\ \nabla & & \nabla \\ t'_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t'_{11}\langle \vec{t}'_{12} \rangle \end{array}$$

4.3 The $\triangleright$ -step occur at the root. We perform the $\rightarrow_{\text{im}}$ step, or not.

$$\begin{array}{ccc} t_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t'_{11}\langle \vec{t}_{12} \rangle & t_{11}\langle \vec{t}_{12} \rangle & \rightarrow_{\text{im}} & t_{11}\langle \vec{t}'_{12} \rangle \\ \nabla & & \nabla & \nabla & & \nabla \\ t_{11} & \rightarrow_{\text{im}} & t'_{11} & t_{11} & = & t_{11} \end{array}$$

5. $t_1 = \{s_1, \dots, s_n\}$. We consider two subcases.

5.1 Both steps occur in the same subterm. It suffices to resort to the IH:

$$\begin{array}{ccc}
\{s_1, \dots, s_i, \dots, s_n\} & \rightarrow_{\text{im}} & \{s_1, \dots, s'_i, \dots, s_n\} \\
\downarrow \nabla & & \downarrow \nabla^+ \\
\{s_1, \dots, s''_i, \dots, s_n\} & \rightarrow_{\text{im}}^= & \{s_1, \dots, s'''_i, \dots, s_n\}
\end{array}$$

5.2 The steps occur in different subterms. Then the diagram can be closed immediately. Indeed, let $i, j \in 1..n$:

$$\begin{array}{ccc}
\{s_1, \dots, s_i, \dots, s_j, \dots, s_n\} & \rightarrow_{\text{im}} & \{s_1, \dots, s'_i, \dots, s_j, \dots, s_n\} \\
\downarrow \nabla & & \downarrow \nabla \\
\{s_1, \dots, s_i, \dots, s'_j, \dots, s_n\} & \rightarrow_{\text{im}} & \{s_1, \dots, s'_i, \dots, s'_j, \dots, s_n\}
\end{array}$$

□